Spring Security

Complete Guide

Beginner → Advanced → Enterprise

- 30 Chapters
- JWT & OAuth2
- Microservices
- Enterprise Patterns

Java 21 | Spring Boot 3.x | Spring Security 6.x

Table of Contents

Chapter 1: Introduction to Security

- Authentication vs Authorization
- SQL Injection / XSS / CSRF
- Spring Security Overview
- Enterprise Security Architecture

Chapter 2: Spring Security Fundamentals

- Security Filter Chain
- SecurityContext & Holder
- Authentication Object
- UserDetailsService
- PasswordEncoder

Chapter 3: Spring Security Configuration

- SecurityFilterChain
- Role-Based Security
- Custom Rules
- PermitAll vs Authenticated
- Multiple Chain Ordering

Chapter 4: Authentication Mechanisms

- HTTP Basic
- Form Login
- Session-Based
- Custom Login & Logout
- Custom AuthenticationProvider

Chapter 5: User Management

- RBAC Schema
- JPA Entities
- UserDetailsService
- Registration & Email Verification
- Admin API

Chapter 6: Password Encoding

- BCrypt / SCrypt / PBKDF2 / Argon2
- DelegatingPasswordEncoder
- Transparent Hash Upgrade
- Password Policy
- History & Expiry

Chapter 7: Authorization Concepts

- hasRole() vs hasAuthority()

- Method-Level Security
- @PreAuthorize / @PostAuthorize
- Custom AuthorizationManager
- RBAC vs ABAC

Chapter 8: JWT Authentication

- JWT Structure
- JwtService Token Engine
- Redis Blacklist
- JwtAuthenticationFilter
- Refresh Token Rotation

Chapter 9: OAuth2 Security

- Authorization Code + PKCE
- Custom OAuth2UserService
- Resource Server
- Client Credentials (M2M)
- Multi-Tenant

Chapter 10: OpenID Connect (OIDC)

- OIDC vs OAuth2
- ID Token Claims
- Discovery Document
- Google Login Integration
- UserInfo Endpoint
- OIDC Logout

Chapter 11: JWT + Microservices Security

- API Gateway Security
- Token Propagation
- Service-to-Service Authentication
- Security in Kafka/Async

Chapter 12: CSRF Protection

- CSRF Attack Anatomy
- CookieCsrfTokenRepository
- SPA Token Flow
- When to Disable
- SameSite Cookies

Chapter 13: CORS Security

- Same-Origin Policy
- Preflight Flow
- Global & Per-Route CORS
- React & Angular Integration
- Dynamic CORS

Chapter 14: Exception Handling

- Security Exception Hierarchy
- AuthenticationEntryPoint
- AccessDeniedHandler
- @RestControllerAdvice
- ErrorResponse DTO

Chapter 15: Spring Security Filters

- Filter Chain Order
- OncePerRequestFilter
- Custom Filter Gallery
- JWT Authentication Filter

Chapter 16: Database Security

- SQL Injection Prevention
- Spring Data JPA Safe Patterns
- Row-Level Security
- Secure Password Storage
- Audit Logging

Chapter 17: HTTPS & SSL / TLS

- SSL/TLS Basics
- Keystore Generation
- Enable HTTPS
- HTTP to HTTPS Redirect
- HSTS
- Certificate Pinning

Chapter 18: Security Headers

- Default Spring Security Headers
- Content Security Policy
- All Headers Reference
- Permissions-Policy

Chapter 19: Method-Level Security (Advanced)

- Enable Method Security
- SpEL Expressions
- Custom Security Expressions
- Service vs Controller Layer

Chapter 20: LDAP Authentication

- LDAP Basics
- Configuration
- Active Directory
- Group-to-Role Mapping
- UserDetailsContextMapper

Chapter 21: Spring Security Testing

- MockMvc Security Tests

- @WithMockUser
- Custom @WithMockUser Principal
- JWT Testing
- Method Security Tests

Chapter 22: Reactive Security (WebFlux)

- Reactive vs Servlet Security
- SecurityWebFilterChain
- ReactiveUserDetailsService
- Current User in WebFlux
- Reactive JWT Filter

Chapter 23: Enterprise Security Patterns

- RBAC
- ABAC
- MFA with TOTP
- SSO via OIDC
- Zero Trust Principles

Chapter 24: Performance & Best Practices

- Stateless Authentication
- Caching UserDetails
- DB Query Optimisation
- Secure Logging
- Performance Checklist

Chapter 25: Frontend Integration

- Token Storage Strategy
- React Secure Auth
- Angular HTTP Interceptor
- React Native Mobile Security

Chapter 26: Logging & Monitoring

- Security Audit Events
- Structured JSON Logging
- Suspicious Activity Detection
- Micrometer Metrics

Chapter 27: Cloud & DevOps Security

- Docker Security
- Kubernetes Secrets
- HashiCorp Vault
- CI/CD Pipeline Security

Chapter 28: Real-Time Security (WebSocket)

- WebSocket Security Challenges
- STOMP Security Config
- Handshake JWT Authentication

- Message-Level @PreAuthorize

Chapter 29: Advanced Topics

- Custom AuthenticationProvider
- Dynamic Role Management
- Virtual Threads + Security
- Security Observability

Chapter 30: Final Enterprise Projects

- Project 1: JWT Auth System
- Project 2: Secure Microservices
- Project 3: Banking Security App
- Learning Path & Tech Stack

Chapter 1

Introduction to Security

1.1 Authentication vs Authorization

Authentication answers **Who are you?** — verifying identity. Authorization answers **What can you do?** — enforcing permissions. In Spring Security, these map to two separate filter chain stages. Authentication failure returns **401 Unauthorized**; authorization failure returns **403 Forbidden**.

```
// Spring Security maps automatically
// AuthenticationException → 401 Unauthorized
// AccessDeniedException → 403 Forbidden

Authentication auth = SecurityContextHolder.getContext().getAuthentication();
String username = auth.getName();
Collection<? extends GrantedAuthority> roles = auth.getAuthorities();
```

1.2 Common Security Threats

SQL Injection

Attackers inject malicious SQL via input fields. Always use parameterized queries or JPA/Spring Data — never string concatenation.

```
// WRONG vs CORRECT
// WRONG — vulnerable to SQL injection
String q = "SELECT * FROM users WHERE username='" + username + "'";

// CORRECT — Spring Data generates safe parameterized queries
Optional&lt;User&gt; findByUsername(String username);

// CORRECT — JPQL with @Param
@Query("SELECT u FROM User u WHERE u.email = :email AND u.active = true")
Optional&lt;User&gt; findActiveByEmail(@Param("email") String email);
```

XSS — Cross-Site Scripting

Attackers inject <script> tags into pages viewed by other users. Defend with Content Security Policy headers, Thymeleaf auto-escaping, and OWASP Java HTML Sanitizer.

```
// Content Security Policy via Spring Security
http.headers(headers -&gt; headers
.contentSecurityPolicy(csp -&gt; csp
.policyDirectives(
"default-src 'self'; script-src 'self'; object-src 'none'")
)
);
```

CSRF — Cross-Site Request Forgery

Tricks authenticated users into submitting malicious requests. Spring Security enables CSRF protection by default for stateful apps. For stateless JWT APIs, CSRF is unnecessary and should be disabled.

```
// Stateful app — CSRF enabled (default)
```

```
http.csrf(csrf -&gt; csrf
.csrfTokenRepository(CookieCsrfTokenRepository.withHttpOnlyFalse())
);

// Stateless JWT API — disable CSRF safely
http.csrf(csrf -&gt; csrf.disable());
```

1.3 Spring Security Filter Chain

Spring Security works as an ordered chain of Servlet filters. Every HTTP request passes through each filter before reaching your controller. Key filters in order: SecurityContextHolderFilter → UsernamePasswordAuthenticationFilter → ExceptionTranslationFilter → AuthorizationFilter.

1.4 Enterprise Security Architecture

```
// Production baseline — SecurityFilterChain
```

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
return http
.sessionManagement(s -&gt; s
.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
.csrf(csrf -&gt; csrf.disable())
.authorizeHttpRequests(auth -&gt; auth
.requestMatchers("/api/auth/**").permitAll()
.requestMatchers("/api/admin/**").hasRole("ADMIN")
.anyRequest().authenticated()
)
.addFilterBefore(jwtAuthFilter(),
UsernamePasswordAuthenticationFilter.class)
.exceptionHandling(ex -&gt; ex
.authenticationEntryPoint(customAuthEntryPoint())
.accessDeniedHandler(customAccessDeniedHandler())
)
.build();
}

@Bean
public PasswordEncoder passwordEncoder() {
return new BCryptPasswordEncoder(12);
}
```

1.5 Quick Reference

Threat	Attack vector	Spring Security Defence
SQL Injection	Malicious SQL in input	JpaRepository, @Param, PreparedStatement
XSS	<script> in output	CSP headers, Thymeleaf escape, Sanitizers
CSRF	Forged cross-site requests	CsrfFilter + CookieCsrfTokenRepository
Auth bypass	Missing auth checks	authorizeHttpRequests + anyRequest().authenticated()
Plain passwords	No hashing	BCryptPasswordEncoder(12)
Info exposure	Stack traces in errors	Custom EntryPoint + AccessDeniedHandler

Chapter 2

Spring Security Fundamentals

2.1 Security Filter Chain

Every HTTP request passes through an ordered list of filters wired by **SecurityFilterChain**. You can define multiple chains ordered with @**Order**.

```
// Multiple filter chains
@Bean @Order(1)
public SecurityFilterChain actuatorChain(HttpSecurity http) throws Exception {
return http
.securityMatcher("/actuator/**")
.authorizeHttpRequests(auth -&gt; auth
.requestMatchers("/actuator/health").permitAll()
.anyRequest().hasRole("MONITORING"))
.httpBasic(Customizer.withDefaults())
.build();
}

@Bean @Order(2)
public SecurityFilterChain apiChain(HttpSecurity http) throws Exception {
return http
.securityMatcher("/api/**")
.csrf(csrf -&gt; csrf.disable())
.sessionManagement(s -&gt; s
.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
.authorizeHttpRequests(auth -&gt; auth
.requestMatchers("/api/auth/**").permitAll()
.anyRequest().authenticated())
.addFilterBefore(jwtAuthFilter,
UsernamePasswordAuthenticationFilter.class)
.build();
}
```

2.2 SecurityContext & SecurityContextHolder

The SecurityContext is a thread-local store holding the authenticated user for the duration of a request. This is how @PreAuthorize and audit logging know who is calling.

```
// Reading the current user — 3 patterns
// Pattern 1 — SecurityContextHolder (works anywhere)
Authentication auth = SecurityContextHolder.getContext().getAuthentication();
String username = auth.getName();

// Pattern 2 — @AuthenticationPrincipal (cleanest in controllers)
@GetMapping("/profile")
public ResponseEntity&lt;UserDto&gt; getProfile(
@AuthenticationPrincipal UserDetails userDetails) { ... }

// Pattern 3 — Custom principal
@GetMapping("/dashboard")
public ResponseEntity&lt;DashboardDto&gt; getDashboard(
@AuthenticationPrincipal CustomUserPrincipal principal) {
return dashboardService.load(principal.getTenantId());
```

```
}
```

2.3 Authentication Object

The Authentication interface carries: principal (who), credentials (proof — cleared after auth), and authorities (what they can do).

```
// Building Authentication in JWT filter
UsernamePasswordAuthenticationToken authentication =
new UsernamePasswordAuthenticationToken(
username, // principal
null, // credentials — null after validation
authorities // GrantedAuthority list
);
authentication.setDetails(
new WebAuthenticationDetailsSource().buildDetails(request));
SecurityContextHolder.getContext().setAuthentication(authentication);
```

2.4 UserDetailsService

The bridge between Spring Security and your user store. Spring calls loadUserByUsername() on every authentication attempt.

```
// Production UserDetailsService with cache
@Service @RequiredArgsConstructor
public class EnterpriseUserDetailsService implements UserDetailsService {

private final UserRepository userRepository;

@Cacheable(value = "userDetails", key = "#email")
@Override
public UserDetails loadUserByUsername(String email)
throws UsernameNotFoundException {
return userRepository.findByEmailWithRoles(email)
.map(this::buildPrincipal)
.orElseThrow(() -&gt;
// Same message for not-found AND wrong-password
// prevents user enumeration
new UsernameNotFoundException("Invalid credentials"));
}

@CacheEvict(value = "userDetails", key = "#email")
public void evictCacheForUser(String email) {}
}
```

2.5 PasswordEncoder

```
// BCrypt strength 12 — production standard
@Bean
public PasswordEncoder passwordEncoder() {
return new BCryptPasswordEncoder(12); // ~400ms per hash
}

// Usage in registration
user.setPassword(passwordEncoder.encode(request.getPassword()));
```

```
// Usage in login (Spring calls this automatically)
passwordEncoder.matches(rawPassword, storedHash); // returns boolean
```

2.6 Architecture Flow

```
HTTP Request
|
v SecurityFilterChain (ordered pipeline)
|
v UsernamePasswordAuthFilter
| calls AuthenticationManager
| delegates to DaoAuthenticationProvider
| uses UserDetailsService --> DB / Cache
| uses PasswordEncoder --> matches()
|
v Authentication object stored in SecurityContextHolder
|
v @PreAuthorize / @AuthenticationPrincipal / audit logs
```

Chapter 3

Spring Security Configuration

3.1 SecurityFilterChain — Enterprise Setup

Define multiple @Order-annotated chains to handle different security domains: Actuator, REST API, and Web MVC.

```
// Three-chain enterprise setup
@Configuration @EnableWebSecurity
@EnableMethodSecurity(prePostEnabled = true)
public class SecurityConfig {

@Bean @Order(1) // Actuator — HTTP Basic
public SecurityFilterChain actuatorChain(HttpSecurity http) throws Exception {
return http.securityMatcher("/actuator/**")
.authorizeHttpRequests(auth -&gt; auth
.requestMatchers("/actuator/health", "/actuator/info").permitAll()
.anyRequest().hasRole("MONITORING"))
.httpBasic(Customizer.withDefaults()).build();
}

@Bean @Order(2) // REST API — JWT stateless
public SecurityFilterChain apiChain(HttpSecurity http) throws Exception {
return http.securityMatcher("/api/**")
.csrf(csrf -&gt; csrf.disable())
.sessionManagement(s -&gt; s
.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
.authorizeHttpRequests(auth -&gt; auth
.requestMatchers(HttpMethod.POST, "/api/auth/**").permitAll()
.requestMatchers("/api/admin/**").hasRole("ADMIN")
.anyRequest().authenticated())
.addFilterBefore(jwtAuthFilter,
UsernamePasswordAuthenticationFilter.class).build();
}
}
```

3.2 Role-Based Security — All Patterns

```
// Complete URL-level rule reference
.authorizeHttpRequests(auth -&gt; auth
.requestMatchers("/public/**").permitAll()
.requestMatchers("/login").anonymous()
.requestMatchers("/profile").authenticated()
.requestMatchers("/change-password").fullyAuthenticated()
.requestMatchers("/admin/**").hasRole("ADMIN")
.requestMatchers("/reports/**").hasAnyRole("ADMIN", "MANAGER")
.requestMatchers("/api/payments/**").hasAuthority("payment:write")
.requestMatchers(HttpMethod.DELETE).hasRole("ADMIN")
.requestMatchers("/internal/**").denyAll()
.anyRequest().authenticated()
)
```

3.3 Method-Level Security

```
// @PreAuthorize — most powerful annotation
@GetMapping("/{id}")
@PreAuthorize("hasRole('ADMIN') or @orderSecurity.isOwner(#id, authentication)")
public OrderDto getById(@PathVariable Long id) { ... }

@PostMapping
@PreAuthorize("hasAuthority('order:create')")
public OrderDto create(@RequestBody CreateOrderRequest req) { ... }

@GetMapping("/invoice/{id}")
@PostAuthorize("returnObject.userId == authentication.principal.userId")
public InvoiceDto getInvoice(@PathVariable Long id) { ... }

@PutMapping("/bulk")
@PreFilter("filterObject.userId == authentication.principal.userId")
public List&lt;OrderDto&gt; bulkUpdate(@RequestBody List&lt;UpdateOrderRequest&gt; orders) { ... }
```

3.4 Role Hierarchy

```
// ADMIN inherits MANAGER inherits USER
@Bean
public RoleHierarchy roleHierarchy() {
RoleHierarchyImpl h = new RoleHierarchyImpl();
h.setHierarchy("""
ROLE_ADMIN &gt; ROLE_MANAGER
ROLE_MANAGER &gt; ROLE_USER
ROLE_USER &gt; ROLE_GUEST
""");
return h;
}
```

3.5 Configuration Cheatsheet

Rule	When to use
permitAll()	Public pages, login, static assets
anonymous()	Login page (redirect if already logged in)
authenticated()	Any logged-in user, any role
fullyAuthenticated()	Sensitive ops — rejects remember-me tokens
hasRole(X)	Coarse-grained role (ROLE_ added automatically)
hasAuthority(x:y)	Fine-grained permission — exact string match
denyAll()	Block path entirely — dev safety net
@PreAuthorize	Per-method rules — most powerful, use this most
@PostAuthorize	Check return value for ownership

Chapter 4

Authentication Mechanisms

4.1 HTTP Basic Authentication

Simplest mechanism — credentials sent as Base64 in every request header. Best for machine-to-machine, internal APIs, and Actuator endpoints.

```
// HTTP Basic config
@Bean @Order(1)
public SecurityFilterChain basicAuthChain(HttpSecurity http) throws Exception {
return http.securityMatcher("/actuator/**", "/internal/**")
.authorizeHttpRequests(auth -&gt; auth
.requestMatchers("/actuator/health").permitAll()
.anyRequest().hasRole("MONITORING"))
.httpBasic(basic -&gt; basic.realmName("Enterprise Monitoring")
.authenticationEntryPoint(new BasicAuthEntryPoint()))
.sessionManagement(s -&gt; s
.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
.build();
}
```

4.2 Form-Based Authentication

```
// Full form login with custom handlers
http.formLogin(form -&gt; form
.loginPage("/login")
.loginProcessingUrl("/login")
.usernameParameter("email")
.defaultSuccessUrl("/dashboard", true)
.successHandler(customSuccessHandler())
.failureHandler(customFailureHandler())
.permitAll()
).logout(logout -&gt; logout
.logoutUrl("/logout")
.logoutSuccessUrl("/login?logout=true")
.invalidateHttpSession(true)
.deleteCookies("JSESSIONID", "remember-me")
);
```

4.3 Session Management

```
// Concurrent session control
http.sessionManagement(session -&gt; session
.sessionCreationPolicy(SessionCreationPolicy.IF_REQUIRED)
.sessionFixation(fix -&gt; fix.changeSessionId())
.maximumSessions(1)
.maxSessionsPreventsLogin(false) // kick old session
.expiredSessionStrategy(customExpiredStrategy())
);

@Bean
public SessionRegistry sessionRegistry() {
return new SessionRegistryImpl(); // required for concurrency control
```

```
}

@Bean
public HttpSessionEventPublisher httpSessionEventPublisher() {
return new HttpSessionEventPublisher();
}
```

4.4 Custom Login — REST JWT Flow

```
// AuthController — login endpoint
@PostMapping("/api/auth/login")
public ResponseEntity&lt;AuthResponse&gt; login(
@RequestBody @Valid LoginRequest req,
HttpServletRequest httpReq) {

if (loginAttemptService.isLocked(req.email()))
throw new TooManyRequestsException("Account locked.");

try {
Authentication auth = authManager.authenticate(
new UsernamePasswordAuthenticationToken(
req.email(), req.password()));

loginAttemptService.recordSuccess(req.email());
CustomUserPrincipal principal =
(CustomUserPrincipal) auth.getPrincipal();

return ResponseEntity.ok(AuthResponse.builder()
.accessToken(jwtService.generateAccessToken(principal))
.refreshToken(refreshTokenService.create(principal))
.tokenType("Bearer")
.expiresIn(jwtService.getAccessTokenExpiry())
.build());

} catch (AuthenticationException ex) {
loginAttemptService.recordFailure(req.email(), getIp(httpReq));
throw new BadCredentialsException("Invalid credentials");
}
}
```

4.5 Mechanism Comparison

Mechanism	Best for	Session?	Token
HTTP Basic	Internal APIs, Actuator, M2M	Stateless	Base64 per request
Form Login	Browser MVC apps	Stateful	JSESSIONID cookie
Session-based	Clustered web apps	Stateful	Spring Session + Redis
JWT / Custom	REST APIs, SPAs	Stateless	Bearer token
API Key	B2B integrations, webhooks	Stateless	X-API-Key header

Chapter 5

User Management

5.1 Database Schema — RBAC

```
-- Full RBAC schema
CREATE TABLE users (
id BIGSERIAL PRIMARY KEY,
email VARCHAR(100) NOT NULL UNIQUE,
password VARCHAR(255) NOT NULL,
tenant_id VARCHAR(50) NOT NULL DEFAULT 'default',
enabled BOOLEAN NOT NULL DEFAULT TRUE,
account_locked BOOLEAN NOT NULL DEFAULT FALSE,
email_verified BOOLEAN NOT NULL DEFAULT FALSE,
failed_attempts INT NOT NULL DEFAULT 0,
created_at TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE TABLE roles (
id BIGSERIAL PRIMARY KEY,
name VARCHAR(50) NOT NULL UNIQUE -- ADMIN, MANAGER, USER
);

CREATE TABLE permissions (
id BIGSERIAL PRIMARY KEY,
name VARCHAR(100) NOT NULL UNIQUE -- order:read, order:write
);

CREATE TABLE role_permissions (
role_id BIGINT REFERENCES roles(id),
permission_id BIGINT REFERENCES permissions(id),
PRIMARY KEY (role_id, permission_id)
);

CREATE TABLE user_roles (
user_id BIGINT REFERENCES users(id),
role_id BIGINT REFERENCES roles(id),
PRIMARY KEY (user_id, role_id)
);
```

5.2 JPA Entities

```
// User entity (abbreviated)
@Entity @Table(name = "users")
@Data @Builder @NoArgsConstructor @AllArgsConstructor
public class User {
@Id @GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;

@Column(unique = true, nullable = false)
private String email;

@Column(nullable = false)
@JsonIgnore // NEVER serialize password
```

```
private String password;

private String tenantId;
private boolean enabled;
private boolean accountLocked;
private boolean emailVerified;
private int failedAttempts;

@ManyToMany(fetch = FetchType.EAGER)
@JoinTable(name = "user_roles",
joinColumns = @JoinColumn(name = "user_id"),
inverseJoinColumns = @JoinColumn(name = "role_id"))
private Set&lt;Role&gt; roles = new HashSet&lt;&gt;();
}
```

5.3 Registration Service

```
// Full registration lifecycle
@Service @RequiredArgsConstructor @Transactional
public class RegistrationService {

public UserResponseDto register(RegisterRequest req) {
if (userRepository.existsByEmail(req.email()))
throw new UserAlreadyExistsException("Email already registered");

Role userRole = roleRepository.findByName("USER").orElseThrow();

User user = User.builder()
.email(req.email())
.password(passwordEncoder.encode(req.password()))
.roles(Set.of(userRole))
.emailVerified(false)
.enabled(true)
.build();

User saved = userRepository.save(user);
sendVerificationEmail(saved); // async
return UserResponseDto.from(saved);
}

public void verifyEmail(String token) {
UserToken t = tokenRepository
.findByTokenAndType(token, VERIFY_EMAIL)
.orElseThrow(() -&gt; new InvalidTokenException("Invalid token"));

if (t.isUsed() || t.isExpired())
throw new InvalidTokenException("Token invalid or expired");

t.getUser().setEmailVerified(true);
t.setUsed(true);
userDetailsService.evictCacheForUser(t.getUser().getEmail());
}
}
```

5.4 Admin User Management API

```
// Role assignment + account lifecycle
```

```
@RestController @RequestMapping("/api/admin/users")
@PreAuthorize("hasRole('ADMIN')")
public class AdminUserController {

@PutMapping("/{id}/roles")
public UserResponseDto updateRoles(@PathVariable Long id,
@RequestBody UpdateRolesRequest req) {
return adminUserService.updateRoles(id, req.roleNames());
}

@PutMapping("/{id}/lock")
public ResponseEntity&lt;Void&gt; lockUser(@PathVariable Long id) {
adminUserService.setLocked(id, true);
return ResponseEntity.noContent().build();
}

@PutMapping("/{id}/expire-credentials")
public ResponseEntity&lt;Void&gt; expireCredentials(@PathVariable Long id) {
adminUserService.expireCredentials(id);
return ResponseEntity.noContent().build();
}

@DeleteMapping("/{id}")
@PreAuthorize("hasAuthority('user:delete')")
public ResponseEntity&lt;Void&gt; deleteUser(@PathVariable Long id) {
adminUserService.delete(id);
return ResponseEntity.noContent().build();
}
}
```

Chapter 6

Password Encoding

6.1 Algorithm Comparison

Algorithm	Memory-hard	Hash time	Use case	Recommendation
BCrypt	Partial	~250ms (cost 12)	General web apps	Default choice
SCrypt	Yes	~300ms	High-value accounts	Financial/healthcare
PBKDF2	No	~200ms (600k)	FIPS-140 compliance	Govt/banking
Argon2id	Yes	~500ms	New apps — best security	PHC winner
MD5 / SHA-1	No	<1ms	Legacy — DO NOT USE	Deprecated

6.2 BCrypt — Deep Dive

```
// BCrypt encoding and matching
@Bean
public PasswordEncoder passwordEncoder() {
// Strength 12 = ~400ms per hash — production recommended
return new BCryptPasswordEncoder(12);
}

// encode() generates a NEW random salt every time
String hash1 = encoder.encode("P@ssword123");
String hash2 = encoder.encode("P@ssword123");
// hash1 != hash2 (different salts) — this is CORRECT

// Format: $2a$12$&lt;22-char salt&gt;&lt;31-char hash&gt;

// matches() extracts salt from stored hash automatically
encoder.matches("P@ssword123", hash1); // true
encoder.matches("wrong", hash1); // false

// upgradeEncoding() — true if cost factor is lower than current
encoder.upgradeEncoding(oldHash); // true if re-hashing needed
```

6.3 DelegatingPasswordEncoder — Migration

The most important encoder for production. Runs multiple encoders simultaneously and upgrades hashes transparently on login — zero downtime, no forced password resets.

```
// Multi-encoder setup
@Bean
public PasswordEncoder passwordEncoder() {
Map&lt;String, PasswordEncoder&gt; encoders = new LinkedHashMap&lt;&gt;();
encoders.put("bcrypt", new BCryptPasswordEncoder(12));
encoders.put("scrypt", SCryptPasswordEncoder.defaultsForSpringSecurity_v5_8());
encoders.put("argon2", Argon2PasswordEncoder.defaultsForSpringSecurity_v5_8());
encoders.put("noop", NoOpPasswordEncoder.getInstance()); // legacy matching
```

```
return new DelegatingPasswordEncoder("bcrypt", encoders);
}

// Stored format: {bcrypt}$2a$12$... {argon2}$argon2id$... {noop}plain
// On login: extracts {id} prefix, delegates to correct encoder
```

6.4 Transparent Hash Upgrade

```
// UserDetailsPasswordService — called after successful login
@Bean
public UserDetailsPasswordService upgradeService() {
return (userDetails, newEncodedPassword) -&gt; {
// Upgrade {noop} or {sha256} hash to {bcrypt} silently
userRepository.updatePassword(
userDetails.getUsername(), newEncodedPassword);
userDetailsService.evictCacheForUser(userDetails.getUsername());
return User.withUserDetails(userDetails)
.password(newEncodedPassword).build();
};
}
```

6.5 Password Policy Enforcement

```
// Composable rule engine
public interface PasswordRule {
PasswordValidationResult validate(String password, String username);
}

// Rules: MinLength, Complexity, UsernameInPassword,
// CommonPasswords, PasswordHistory (last 5)

@Service
public class PasswordPolicyService {
private final List&lt;PasswordRule&gt; rules; // Spring injects all

public void validate(String password, String username) {
List&lt;String&gt; violations = rules.stream()
.map(r -&gt; r.validate(password, username))
.filter(r -&gt; !r.isValid())
.map(PasswordValidationResult::getMessage)
.collect(Collectors.toList());

if (!violations.isEmpty())
throw new PasswordPolicyViolationException(violations);
}
}
```

6.6 Enterprise Password Checklist

- Minimum 12 characters
- Complexity: uppercase + lowercase + digit + special char
- Reject username embedded in password
- Block top-10,000 common passwords
- History check: reject last 5 passwords
- Expiry: 90 days + 7-day warning email

- Pepper (PBKDF2) for extra server-side secret
- DelegatingPasswordEncoder for zero-downtime migration

Chapter 7

Authorization Concepts

7.1 hasRole() vs hasAuthority()

hasRole("ADMIN") internally checks for authority string **ROLE_ADMIN** — the ROLE_ prefix is added automatically. **hasAuthority("order:write")** checks the exact string — no prefix added. Use roles for coarse-grained access, authorities for fine-grained permissions.

```
// URL-level — side by side
.authorizeHttpRequests(auth -&gt; auth
.requestMatchers("/admin/**").hasRole("ADMIN") // checks ROLE_ADMIN
.requestMatchers("/api/export").hasAuthority("report:export") // exact match
.requestMatchers("/reports/**").hasAnyRole("ADMIN", "MANAGER")
.requestMatchers("/api/finance/**")
.access(new WebExpressionAuthorizationManager(
"hasRole('MANAGER') and hasAuthority('finance:read')"))
.anyRequest().authenticated()
)
```

7.2 Method Security — Complete Reference

```
// All annotations
// @PreAuthorize — before method, SpEL expressions
@PreAuthorize("hasRole('ADMIN') or @docSecurity.isOwner(#id, authentication)")
public DocumentDto update(@PathVariable Long id, ...) { ... }

// @PostAuthorize — after method, return value available
@PostAuthorize("returnObject.ownerId == authentication.principal.userId")
public InvoiceDto getInvoice(Long id) { ... }

// @PreFilter — filter input collection before invocation
@PreFilter("filterObject.ownerId == authentication.principal.userId")
public List&lt;DocumentDto&gt; bulkUpdate(List&lt;UpdateRequest&gt; requests) { ... }

// @PostFilter — filter output collection after return
@PostFilter("filterObject.tenantId == authentication.principal.tenantId")
public List&lt;DocumentDto&gt; findAll() { ... }

// @Secured — simple, no SpEL
@Secured({"ROLE_ADMIN", "ROLE_EDITOR"})
public DocumentDto publish(Long id) { ... }
```

7.3 Custom Security Bean — Ownership Checks

```
// @docSecurity bean — used in @PreAuthorize SpEL
@Component("docSecurity")
@RequiredArgsConstructor
public class DocumentSecurityService {

public boolean isOwner(Long docId, Authentication auth) {
CustomUserPrincipal p = (CustomUserPrincipal) auth.getPrincipal();
return documentRepository.findById(docId)
.map(doc -&gt; doc.getOwnerId().equals(p.getUserId()))
```

```
.orElse(false);
}

public boolean isSameTenant(Long docId, Authentication auth) {
CustomUserPrincipal p = (CustomUserPrincipal) auth.getPrincipal();
return documentRepository.findById(docId)
.map(doc -&gt; doc.getTenantId().equals(p.getTenantId()))
.orElse(false);
}
}
```

7.4 ABAC — Policy Engine

Attribute-Based Access Control evaluates Subject + Resource + Action + Environment attributes. Best for multi-tenant SaaS, compliance-heavy systems, and dynamic security rules.

```
// Policy engine — first-match wins
@Service
public class PolicyEngine {
private final List&lt;AccessPolicy&gt; policies; // Spring injects all

public PolicyDecision evaluate(PolicyRequest req) {
return policies.stream()
.sorted(Comparator.comparingInt(AccessPolicy::priority))
.map(p -&gt; p.evaluate(req))
.filter(d -&gt; d != PolicyDecision.ABSTAIN)
.findFirst()
.orElse(PolicyDecision.DENY); // default deny
}
}

// Policies: AdminOverride(1), TenantIsolation(2), MfaRequired(5),
// DepartmentScope(10), BusinessHours(15), ClearanceLevel(20)
```

7.5 Authorization Layers

Layer	Mechanism	Granularity	Best for
URL Rules	authorizeHttpRequests	Coarse — path/method	Fast, first-line gate
Method Security	@PreAuthorize	Per-method + SpEL	Business logic control
Domain Object	@Bean SpEL / ACL	Per-object ownership	Tenant / ownership checks
ABAC Policy	PolicyEngine + AOP	Full context-aware	Compliance, dynamic rules

Chapter 8

JWT Authentication

8.1 JWT Structure

A JWT has three Base64URL-encoded parts separated by dots: **Header** (algorithm + type), **Payload** (claims), **Signature** (integrity proof). The payload is NOT encrypted — never put passwords or secrets in claims.

Part	Content	Key claims
Header	Algorithm + token type	alg (HS256/RS256), typ, kid
Payload	Claims — base64 encoded	sub, iss, aud, exp, iat, jti, roles
Signature	HMAC or RSA of header+payload	Verifies token wasn't tampered

8.2 JwtService — Core Engine

```
// Token generation
public String generateAccessToken(CustomUserPrincipal principal) {
return Jwts.builder()
.subject(String.valueOf(principal.getUserId()))
.issuer(props.getIssuer())
.audience().add(props.getAudience()).and()
.issuedAt(Date.from(Instant.now()))
.expiration(Date.from(Instant.now()
.plusSeconds(props.getAccessTokenExpiry())))
.id(UUID.randomUUID().toString()) // jti for blacklisting
.claim("email", principal.getEmail())
.claim("tenantId", principal.getTenantId())
.claim("roles", extractRoles(principal))
.claim("type", "ACCESS")
.signWith(secretKey) // HS256 / RS256
.compact();
}

// Token validation
public Claims validateAndExtractClaims(String token) {
return Jwts.parser()
.verifyWith(secretKey)
.requireIssuer(props.getIssuer())
.requireAudience(props.getAudience())
.build()
.parseSignedClaims(token)
.getPayload();
// Throws: ExpiredJwtException, MalformedJwtException,
// SecurityException (bad signature)
}
```

8.3 Token Blacklist — Redis

```
// Add to blacklist on logout
@Service
public class TokenBlacklistService {
```

```
public void blacklist(String jti, Instant expiry) {
long ttl = Duration.between(Instant.now(), expiry).getSeconds() + 60;
if (ttl &gt; 0)
redisTemplate.opsForValue().set(
"jwt:blacklist:" + jti, "revoked", Duration.ofSeconds(ttl));
}

public boolean isBlacklisted(String jti) {
return Boolean.TRUE.equals(
redisTemplate.hasKey("jwt:blacklist:" + jti));
}
}
```

8.4 Refresh Token — Rotation + Reuse Detection

```
// Rotate refresh token on every use
public RefreshTokenRecord validateAndGet(String tokenValue) {
RefreshTokenRecord record = tokenRepository.findByToken(tokenValue)
.orElseThrow(() -&gt; new InvalidTokenException("Not found"));

if (record.isRevoked()) {
// Token reuse detected — possible theft
revokeAll(record.getUserId()); // nuclear option
throw new InvalidTokenException("Token reuse detected");
}
if (record.getExpiresAt().isBefore(Instant.now()))
throw new InvalidTokenException("Refresh token expired");

return record;
}

public String rotate(RefreshTokenRecord old) {
old.setRevoked(true); // invalidate old token
tokenRepository.save(old);
return create(loadPrincipal(old.getUserId())); // issue new
}
```

8.5 JWT Security Checklist

Setting	Value	Reason
Access token TTL	15 minutes	Short-lived = less damage if leaked
Refresh token TTL	7 days	Balance UX vs security
Algorithm (dev)	HS256	Simpler, symmetric
Algorithm (prod)	RS256	Asymmetric — no secret sharing
Key size (RS256)	2048 bits	NIST minimum recommendation
Token size target	< 1 KB	Keep claims minimal
jti claim	UUID per token	Required for blacklisting
iss + aud validate	Always	Prevents token reuse across services

Chapter 9

OAuth2 Security

9.1 Grant Types

Grant Type	When to use	Example
Auth Code + PKCE	Web apps + SPAs — user logs in via browser	Google login
Client Credentials	M2M — service calls service, no user	Order → Inventory
Refresh Token	Silent token renewal	Auto-refresh
Device Code	Smart TV, CLI, IoT	TV streaming app
Resource Owner Pwd	Legacy only — avoid	Trusted 1st-party

9.2 OAuth2 Login Configuration

```
// Spring auto-handles the full auth code flow
http.oauth2Login(oauth2 -&gt; oauth2
.loginPage("/login")
.authorizationEndpoint(ep -&gt; ep
.baseUri("/oauth2/authorize")
.authorizationRequestRepository(cookieAuthorizationRequestRepository()))
.userInfoEndpoint(ui -&gt; ui
.userService(oAuth2UserService) // OAuth2 providers (GitHub)
.oidcUserService(oidcUserService())) // OIDC providers (Google, Keycloak)
.successHandler(successHandler)
.failureHandler(failureHandler)
);
```

9.3 Custom OAuth2UserService — User Provisioning

```
// Create or update user on first social login
@Service
public class CustomOAuth2UserService extends DefaultOAuth2UserService {

@Override @Transactional
public OAuth2User loadUser(OAuth2UserRequest request) {
OAuth2User oAuth2User = super.loadUser(request);
String provider = request.getClientRegistration().getRegistrationId();
String email = extractEmail(oAuth2User, provider);
String providerId = extractProviderId(oAuth2User, provider);

User user = userRepository
.findByProviderAndProviderId(provider, providerId)
.map(existing -&gt; updateExistingUser(existing, oAuth2User))
.orElseGet(() -&gt; registerNewUser(
provider, providerId, email, oAuth2User));

return new OAuth2UserPrincipal(user, oAuth2User.getAttributes());
}
}
```

9.4 Resource Server — JWT Validation

```
// Auto-validates tokens from external Auth Server
http.oauth2ResourceServer(oauth2 -&gt; oauth2
.jwt(jwt -&gt; jwt
// JwtDecoder auto-configured from jwk-set-uri in application.yml
.jwtAuthenticationConverter(jwtAuthConverter)
)
.authenticationEntryPoint(customEntryPoint())
.accessDeniedHandler(customAccessDeniedHandler())
);

// application.yml
spring.security.oauth2.resourceserver.jwt:
jwk-set-uri: https://auth.yourapp.com/.well-known/jwks.json
issuer-uri: https://auth.yourapp.com/realms/myrealm
```

9.5 Client Credentials — M2M

```
// WebClient auto-fetches and caches tokens
@Bean
public WebClient inventoryWebClient(
OAuth2AuthorizedClientManager manager) {

ServletOAuth2AuthorizedClientExchangeFilterFunction oauth2 =
new ServletOAuth2AuthorizedClientExchangeFilterFunction(manager);
oauth2.setDefaultClientRegistrationId("internal-service");

return WebClient.builder()
.baseUrl("https://inventory-service.internal")
.apply(oauth2.oauth2Configuration())
.build();
}

// Usage — token automatically injected as Bearer header
InventoryDto item = inventoryWebClient.get()
.uri("/api/inventory/{id}", productId)
.retrieve()
.bodyToMono(InventoryDto.class)
.block();
```

Chapter 10

OpenID Connect (OIDC)

10.1 OIDC Basics

OpenID Connect is an identity layer built on top of OAuth2. Where OAuth2 handles **authorisation** (access to resources), OIDC handles **authentication** (verifying who the user is). OIDC adds an **ID Token** (a JWT) alongside the OAuth2 access token, and a standard **UserInfo** endpoint for fetching profile data.

Concept	OAuth2	OIDC addition
Purpose	Authorise access to resources	Authenticate the user's identity
Token	access_token (opaque or JWT)	id_token (always JWT)
Scopes	Custom (read, write…)	openid, profile, email, address, phone
User info	Not standardised	/userinfo endpoint (standard)
Discovery	None	/.well-known/openid-configuration

10.2 ID Token — Structure & Claims

```
// ID Token is always a signed JWT
// Header
{ "alg": "RS256", "kid": "key-id-1", "typ": "JWT" }

// Payload — standard OIDC claims
{
"iss": "https://accounts.google.com", // issuer
"sub": "1234567890", // unique user ID at provider
"aud": "your-client-id", // your app
"exp": 1716239022, // expiry
"iat": 1716235422, // issued at
"nonce": "abc123", // replay protection

// Profile claims (scope=profile)
"name": "Alice Smith",
"given_name": "Alice",
"family_name": "Smith",
"picture": "https://lh3.googleusercontent.com/...",

// Email claims (scope=email)
"email": "alice@example.com",
"email_verified": true
}
```

10.3 Discovery Document

```
// Auto-discover provider config — no hardcoding URLs
# Spring uses issuer-uri to auto-fetch /.well-known/openid-configuration
spring:
security:
oauth2:
```

```
client:
provider:
google:
issuer-uri: https://accounts.google.com
keycloak:
issuer-uri: https://auth.yourapp.com/realms/myrealm

# Spring fetches:
# https://accounts.google.com/.well-known/openid-configuration
# → authorization_endpoint, token_endpoint, jwks_uri, userinfo_endpoint
# → all wired automatically — zero manual URL config
```

10.4 Google Login Integration

```
// application.yml — Google OIDC login
```

```
spring:
security:
oauth2:
client:
registration:
google:
client-id: ${GOOGLE_CLIENT_ID}
client-secret: ${GOOGLE_CLIENT_SECRET}
scope:
- openid # required for OIDC
- profile # name, picture
- email # email, email_verified
redirect-uri: "{baseUrl}/login/oauth2/code/{registrationId}"
provider:
google:
issuer-uri: https://accounts.google.com
```

```
// Custom OIDC user service — provision user on first login
```

```
@Service
@RequiredArgsConstructor
public class CustomOidcUserService extends OidcUserService {

private final UserRepository userRepository;
private final RoleRepository roleRepository;

@Override @Transactional
public OidcUser loadUser(OidcUserRequest request)
throws OAuth2AuthenticationException {

OidcUser oidcUser = super.loadUser(request); // validates ID token
OidcIdToken idToken = oidcUser.getIdToken();

// Extract standard claims from ID Token
String sub = idToken.getSubject(); // unique at provider
String email = idToken.getEmail();
String name = idToken.getFullName();
String picture = idToken.getClaim("picture");
boolean verified = Boolean.TRUE.equals(idToken.getEmailVerified());
String provider = request.getClientRegistration().getRegistrationId();

User user = userRepository.findByProviderAndProviderId(provider, sub)
.map(u -> updateUser(u, name, picture))
```

```
.orElseGet(() -> createUser(sub, email, name, picture,
verified, provider));

return new OidcUserPrincipal(user, oidcUser.getAttributes(),
oidcUser.getIdToken(),
oidcUser.getUserInfo());
}

private User createUser(String sub, String email, String name,
String picture, boolean verified, String provider) {
Role defaultRole = roleRepository.findByName("USER").orElseThrow();
return userRepository.save(User.builder()
.providerId(sub).provider(provider)
.email(email).username(email)
.firstName(extractFirst(name)).lastName(extractLast(name))
.imageUrl(picture).emailVerified(verified)
.enabled(true).roles(Set.of(defaultRole)).build());
}

private User updateUser(User u, String name, String picture) {
u.setFirstName(extractFirst(name));
u.setLastName(extractLast(name));
u.setImageUrl(picture);
return userRepository.save(u);
}
}
```

10.5 UserInfo Endpoint

```
// Fetch additional claims from UserInfo endpoint
@Service
@RequiredArgsConstructor
public class OidcUserInfoService {

private final OAuth2AuthorizedClientService clientService;

// Call UserInfo endpoint to get fresh profile data
public Map<String, Object> getUserInfo(String clientRegistrationId,
String principalName) {
OAuth2AuthorizedClient client =
clientService.loadAuthorizedClient(
clientRegistrationId, principalName);

if (client == null) return Collections.emptyMap();

return WebClient.create()
.get()
.uri(client.getClientRegistration()
.getProviderDetails()
.getUserInfoEndpoint()
.getUri())
.headers(h -> h.setBearerAuth(
client.getAccessToken().getTokenValue()))
.retrieve()
.bodyToMono(new ParameterizedTypeReference<Map<String,Object>>(){})
.block();
}
```

```
}
```

10.6 OIDC Logout — RP-Initiated

```
// Log out from both your app AND the identity provider
@Controller
@RequiredArgsConstructor
public class OidcLogoutController {

@GetMapping("/logout/oidc")
public String oidcLogout(HttpServletRequest request,
Authentication authentication,
@RegisteredOAuth2AuthorizedClient("google")
OAuth2AuthorizedClient client) {

// 1. Clear local session
request.getSession().invalidate();
SecurityContextHolder.clearContext();

// 2. Redirect to provider's end_session_endpoint
String idTokenHint = ((OidcUser) authentication.getPrincipal())
.getIdToken().getTokenValue();

String endSessionUri = UriComponentsBuilder
.fromUriString("https://accounts.google.com/o/oauth2/revoke")
.queryParam("token", idTokenHint)
.queryParam("post_logout_redirect_uri", "https://yourapp.com/")
.build().toUriString();

return "redirect:" + endSessionUri;
}
}
```

10.7 OIDC vs OAuth2 Quick Reference

Feature	Use OAuth2	Use OIDC
Know WHO the user is	✗	✓ ID Token + sub claim
Verify email is confirmed	✗	✓ email_verified claim
Get profile (name, picture)	✗	✓ profile scope
Single Sign-On (SSO)	✗	✓ session_state + iframe
Access third-party API	✓	Use access_token
Token introspection	✓	✓ Both support it

Chapter 11

JWT + Microservices Security

11.1 Architecture Overview

The API Gateway is the single security perimeter. It validates JWT tokens, rate-limits requests, extracts user context, and forwards trusted headers to downstream services. Downstream services trust gateway headers — no JWT re-validation needed.

```
# application.yml — API Gateway routes
```

```
spring.cloud.gateway.routes:
- id: order-service
uri: lb://order-service
predicates:
- Path=/api/orders/**
filters:
- AuthenticationFilter # validates JWT
- name: RequestRateLimiter
args:
redis-rate-limiter.replenishRate: 100
redis-rate-limiter.burstCapacity: 200
- AddRequestHeader=X-Gateway-Source, api-gateway
- RemoveRequestHeader=Authorization # strip before forwarding
```

11.2 Gateway Authentication Filter

```
// Validates JWT, injects user context headers
```

```
@Component
public class AuthenticationFilter implements GlobalFilter, Ordered {
@Override
public Mono&lt;Void&gt; filter(ServerWebExchange exchange, GatewayFilterChain chain) {
String token = extractToken(exchange.getRequest());
if (token == null) return chain.filter(exchange);

return jwtDecoder.decode(token)
.flatMap(jwt -&gt; {
if (blacklistService.isBlacklisted(jwt.getId()))
return unauthorized(exchange, "Token revoked");

// Inject user context as trusted headers
ServerHttpRequest enriched = exchange.getRequest().mutate()
.header("X-User-Id", jwt.getSubject())
.header("X-User-Email", jwt.getClaimAsString("email"))
.header("X-Tenant-Id", jwt.getClaimAsString("tenantId"))
.header("X-User-Roles", String.join(",", extractRoles(jwt)))
.header("X-Request-Id", UUID.randomUUID().toString())
.build();
return chain.filter(exchange.mutate().request(enriched).build());
});
}
@Override public int getOrder() { return -100; }
}
```

11.3 Token Propagation — Downstream Services

```
// GatewaySecurityContextFilter — reads injected headers
@Component
public class GatewaySecurityContextFilter extends OncePerRequestFilter {

@Override
protected void doFilterInternal(HttpServletRequest req, ...) {
String userId = req.getHeader("X-User-Id");
String tenantId = req.getHeader("X-Tenant-Id");
String roles = req.getHeader("X-User-Roles");

if (StringUtils.hasText(userId)) {
List&lt;GrantedAuthority&gt; authorities = Arrays.stream(roles.split(","))
.map(r -&gt; new SimpleGrantedAuthority("ROLE_" + r.trim()))
.collect(Collectors.toList());

CustomUserPrincipal principal = CustomUserPrincipal.builder()
.userId(Long.parseLong(userId))
.tenantId(tenantId)
.authorities(authorities)
.build();

SecurityContextHolder.getContext().setAuthentication(
new UsernamePasswordAuthenticationToken(
principal, null, authorities));
}
chain.doFilter(req, response);
}
}
```

11.4 Service-to-Service Authentication

```
// Internal JWT — 30-second TTL
public String generateServiceToken(String targetService,
CustomUserPrincipal userCtx) {
return Jwts.builder()
.subject(serviceName) // calling service
.audience().add(targetService).and()
.expiration(Date.from(Instant.now().plusSeconds(30)))
.id(UUID.randomUUID().toString())
.claim("type", "SERVICE")
.claim("originalUserId", userCtx.getUserId()) // propagate user
.claim("originalTenantId",userCtx.getTenantId())
.signWith(internalSigningKey)
.compact();
}

// Interceptor adds S2S token to every outbound RestClient call
public ClientHttpResponse intercept(HttpRequest req, ...) {
String token = tokenService.generateServiceToken(
extractTargetService(req.getURI()), extractCurrentUser());
req.getHeaders().set("X-Service-Token", token);
return execution.execute(req, body);
}
```

11.5 Request Flow Summary

```
Client --JWT--> API Gateway
| validate JWT (signature + expiry + blacklist)
| rate-limit (Redis sliding window)
| inject headers (X-User-Id, X-Tenant-Id, X-User-Roles)
| strip Authorization header
v
Order Service (GatewayContextFilter)
| read headers → SecurityContext (no JWT parsing)
| @PreAuthorize works normally
v calls with X-Service-Token (30s)
Inventory Service (InternalServiceAuthFilter)
| validate X-Service-Token (caller trusted)
| restore original user context
| tenant-scoped DB queries
```

11.6 S2S Pattern Comparison

Pattern	How	Pros	Cons
OAuth2 Client Credentials	Fetch token from Auth Server	Standard, short TTL, auto-refresh	Auth Server dependency
Internal JWT	Sign with shared secret	Fast, no external call	Secret rotation complexity
mTLS	Both services present certs	Strongest, no shared secrets	PKI + service mesh required

Chapter 12

CSRF Protection

12.1 How CSRF Attacks Work

Cross-Site Request Forgery tricks an authenticated user's browser into sending a malicious request to your server. Because the browser automatically attaches cookies (including session cookies), the server cannot distinguish the legitimate request from the forged one. CSRF only applies to stateful (session-based) applications — JWT in Authorization headers is immune because browsers do not send headers automatically.

```
// Attack scenario — bank transfer forgery
<!-- Malicious page on evil.com — user visits while logged in to bank.com -->
<html>
<body onload="document.forms[0].submit()">
<form method="POST" action="https://bank.com/transfer">
<input name="to" value="attacker-account" />
<input name="amount" value="9999" />
</form>
</body>
</html>
<!-- Browser auto-sends bank.com session cookie → transfer executes! -->
```

12.2 Spring Security CSRF Tokens

```
// CSRF enabled by default for stateful apps
// Spring Security adds CsrfFilter automatically
// It generates a unique token per session and validates it on state-changing requests

// Default — stores token in HTTP session
http.csrf(Customizer.withDefaults());

// SPA (React/Angular) — store token in cookie so JS can read it
http.csrf(csrf -> csrf
.csrfTokenRepository(CookieCsrfTokenRepository.withHttpOnlyFalse())
// httpOnly=false lets JavaScript read the XSRF-TOKEN cookie
);

// Custom token repository — e.g. store in Redis for distributed apps
http.csrf(csrf -> csrf
.csrfTokenRepository(redisCsrfTokenRepository())
.sessionAuthenticationStrategy(new CsrfAuthenticationStrategy(
redisCsrfTokenRepository()))
);
```

```
// How the token flow works
// 1. User makes GET /login → Spring sends XSRF-TOKEN cookie
// Set-Cookie: XSRF-TOKEN=abc123; Path=/; SameSite=Strict

// 2. SPA reads cookie value using JS:
const csrfToken = document.cookie
.split('; ')
.find(row => row.startsWith('XSRF-TOKEN='))
?.split('=')[1];
```

```
// 3. SPA sends token in header on every state-changing request:
fetch('/api/transfer', {
method: 'POST',
headers: { 'X-XSRF-TOKEN': csrfToken,
'Content-Type': 'application/json' },
body: JSON.stringify({ to: 'bob', amount: 100 })
});

// 4. CsrfFilter compares header token vs session token → match = allow
// Attacker cannot read the cookie (SOP blocks cross-origin reads)
```

12.3 Thymeleaf CSRF Integration

```
// Thymeleaf auto-inserts CSRF token in forms
```

```
<!-- Thymeleaf form — _csrf hidden field added automatically -->
<form th:action="@{/transfer}" method="post">
<input type="text" name="amount" />
<button type="submit">Transfer</button>
<!-- Spring adds: <input type="hidden" name="_csrf" value="abc123"/> -->
</form>

<!-- Manual if needed -->
<input type="hidden" th:name="${_csrf.parameterName}"
th:value="${_csrf.token}"/>
```

12.4 When to Disable CSRF

```
// Safe to disable ONLY for stateless JWT APIs
```

```
// SAFE — stateless REST API with JWT in Authorization header
// Reason: browser never auto-sends Authorization header
// So CSRF attack cannot include the JWT → forged requests are rejected
http.csrf(csrf -> csrf.disable())
.sessionManagement(s -> s
.sessionCreationPolicy(SessionCreationPolicy.STATELESS));

// SAFE — disable only for specific paths (e.g. webhook receivers)
http.csrf(csrf -> csrf
.ignoringRequestMatchers("/api/webhooks/**") // signed with HMAC
);

// UNSAFE — never disable CSRF for form-based login without replacement
// http.csrf(csrf -> csrf.disable()) ← if you use sessions + cookies
```

12.5 SameSite Cookie — Modern CSRF Defence

```
// SameSite=Strict prevents CSRF at cookie level
```

```
# application.yml — set SameSite on session cookie
server:
servlet:
session:
cookie:
same-site: strict # or 'lax' for slightly broader compatibility
http-only: true # JavaScript cannot read JSESSIONID
secure: true # HTTPS only

# SameSite=Strict: cookie NOT sent on cross-site requests at all
```

```
# → CSRF impossible even without CSRF tokens
# SameSite=Lax: sent on top-level navigation only
# → protects against most CSRF, allows OAuth2 redirect flows
```

12.6 CSRF Checklist

App type	CSRF needed?	Strategy
Traditional MVC (sessions + cookies)	Yes	CookieCsrfTokenRepository, Thymeleaf auto
SPA + JWT in Authorization header	No	Disable CSRF safely
SPA + HttpOnly cookie JWT	Yes	CookieCsrfTokenRepository + X-XSRF-TOKEN header
REST API with basic auth	No	Stateless — disable
Mobile app	No	Cannot be tricked by browser
Webhook receiver	No	Use HMAC signature instead

Chapter 13

CORS Security

13.1 Same-Origin Policy

Origin = scheme + host + port — all three must match. The SOP is enforced by browsers, not servers. It blocks cross-origin fetch/XHR responses but allows image/script/CSS loads and form submissions. CORS is the server-side mechanism to selectively relax the SOP.

```
// Origin examples
// Same origin (both https://app.yourapp.com:443)
https://app.yourapp.com/page1 → https://app.yourapp.com/api SAME

// Different origin:
https://app.yourapp.com → http://app.yourapp.com DIFFERENT (scheme)
https://app.yourapp.com → https://api.yourapp.com DIFFERENT (host)
https://app.yourapp.com → https://app.yourapp.com:8080 DIFFERENT (port)
```

13.2 Global CORS Configuration

```
// Production CORS setup
@Bean
public CorsConfigurationSource corsConfigurationSource() {
CorsConfiguration config = new CorsConfiguration();

// NEVER use wildcard with allowCredentials = true
config.setAllowedOrigins(List.of(
"https://app.yourapp.com",
"https://dashboard.yourapp.com"
));
config.setAllowedMethods(List.of(
"GET","POST","PUT","PATCH","DELETE","OPTIONS"));
config.setAllowedHeaders(List.of(
"Authorization","Content-Type","X-Tenant-Id","X-Request-Id"));
config.setExposedHeaders(List.of(
"X-Total-Count","X-Page-Number","X-Request-Id"));
config.setAllowCredentials(true);
config.setMaxAge(3600L); // cache preflight 1 hour

UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
source.registerCorsConfiguration("/**", config);
return source;
}

// Wire into security chain — CORS processed before CSRF
http.cors(cors -&gt; cors.configurationSource(corsConfigurationSource()))
.csrf(csrf -&gt; csrf.disable());

// Always permit OPTIONS before any auth rules
.requestMatchers(HttpMethod.OPTIONS, "/**").permitAll()
```

13.3 CORS Security Pitfalls

Pitfall	Wrong	Fix
Wildcard + credentials	allowedOrigins(*) + credentials=true	List explicit origins only
Reflect Origin blindly	response.setHeader(request.getHeader('Origin'))	Validate against allowlist
Missing Vary: Origin	No Vary header (CDN caches wrong origin)	Spring adds automatically — don't strip
null origin	allowedOrigins('null')	Never add null to allowed list
OPTIONS gets 401	No permit for OPTIONS method	permitAll() for OPTIONS /**

13.4 React Frontend Integration

```
// axios instance with CORS + JWT + auto-refresh
const api = axios.create({
baseURL: process.env.REACT_APP_API_URL,
withCredentials: true, // required when server uses Allow-Credentials: true
timeout: 10_000,
headers: { 'Content-Type': 'application/json', 'X-Api-Version': 'v1' },
});

// Request interceptor — attach JWT
api.interceptors.request.use(config =&gt; {
const token = localStorage.getItem('accessToken');
if (token) config.headers.Authorization = `Bearer ${token}`;
return config;
});

// Response interceptor — handle 401, refresh token
api.interceptors.response.use(
response =&gt; response,
async error =&gt; {
if (error.response?.status === 401 &amp;&amp; !error.config._retry) {
error.config._retry = true;
const { data } = await axios.post('/api/auth/refresh',
{ refreshToken: localStorage.getItem('refreshToken') });
localStorage.setItem('accessToken', data.accessToken);
error.config.headers.Authorization = `Bearer ${data.accessToken}`;
return api(error.config);
}
return Promise.reject(error);
}
);
```

13.5 Angular HTTP Interceptor

```
// Angular auth interceptor
@Injectable()
export class AuthInterceptor implements HttpInterceptor {

intercept(req: HttpRequest&lt;unknown&gt;,
next: HttpHandler): Observable&lt;HttpEvent&lt;unknown&gt;&gt; {

const token = this.authService.getAccessToken();
const enriched = req.clone({
setHeaders: token
```

```
? { Authorization: `Bearer ${token}`, 'X-Api-Version': 'v1' }
: {},
withCredentials: true,
});

return next.handle(enriched).pipe(
catchError((err: HttpErrorResponse) =&gt; {
if (err.status === 401)
return this.handle401(enriched, next);
return throwError(() =&gt; err);
})
);
}
}
```

Chapter 14

Exception Handling

14.1 Spring Security Exception Hierarchy

```
// Two root exception types
// AuthenticationException → 401 Unauthorized
// ■■■ BadCredentialsException wrong password
// ■■■ UsernameNotFoundException user not found
// ■■■ AccountExpiredException account expired
// ■■■ CredentialsExpiredException password expired
// ■■■ DisabledException account disabled
// ■■■ LockedException account locked
// ■■■ InsufficientAuthenticationException not fully auth'd

// AccessDeniedException → 403 Forbidden
// ■■■ thrown when authenticated but missing required authority

// ExceptionTranslationFilter catches both and delegates:
// AuthenticationException → AuthenticationEntryPoint (401)
// AccessDeniedException → AccessDeniedHandler (403)
```

14.2 Custom AuthenticationEntryPoint

```
// JSON 401 response instead of redirect to login page
@Component
@RequiredArgsConstructor
public class CustomAuthEntryPoint implements AuthenticationEntryPoint {

private final ObjectMapper objectMapper;

@Override
public void commence(HttpServletRequest request,
HttpServletResponse response,
AuthenticationException ex) throws IOException {

response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
response.setContentType(MediaType.APPLICATION_JSON_VALUE);
response.setCharacterEncoding("UTF-8");

String code = resolveCode(ex);

objectMapper.writeValue(response.getOutputStream(), Map.of(
"status", 401,
"code", code,
"message", sanitize(ex),
"path", request.getServletPath(),
"timestamp", Instant.now().toString()
));
}

private String resolveCode(AuthenticationException ex) {
if (ex instanceof BadCredentialsException) return "INVALID_CREDENTIALS";
if (ex instanceof LockedException) return "ACCOUNT_LOCKED";
```

```
if (ex instanceof DisabledException) return "ACCOUNT_DISABLED";
if (ex instanceof AccountExpiredException) return "ACCOUNT_EXPIRED";
if (ex instanceof CredentialsExpiredException) return "PASSWORD_EXPIRED";
return "UNAUTHORIZED";
}

private String sanitize(AuthenticationException ex) {
// Never expose internal details
if (ex instanceof LockedException) return "Account is locked";
if (ex instanceof DisabledException) return "Account is disabled";
if (ex instanceof BadCredentialsException) return "Invalid credentials";
if (ex instanceof AccountExpiredException) return "Account has expired";
return "Authentication required";
}
}
```

14.3 Custom AccessDeniedHandler

```
// JSON 403 response with audit logging
@Component
@RequiredArgsConstructor
@Slf4j
public class CustomAccessDeniedHandler implements AccessDeniedHandler {

private final AuditService auditService;

@Override
public void handle(HttpServletRequest request,
HttpServletResponse response,
AccessDeniedException ex) throws IOException {

Authentication auth = SecurityContextHolder.getContext()
.getAuthentication();

log.warn("ACCESS DENIED | user={} | path={} | method={}",
auth != null ? auth.getName() : "anonymous",
request.getServletPath(), request.getMethod());

// Audit the denied access attempt
auditService.recordDenied(
auth != null ? auth.getName() : "anonymous",
request.getServletPath(), request.getMethod(),
request.getRemoteAddr());

response.setStatus(HttpServletResponse.SC_FORBIDDEN);
response.setContentType(MediaType.APPLICATION_JSON_VALUE);

new ObjectMapper().writeValue(response.getOutputStream(), Map.of(
"status", 403,
"code", "ACCESS_DENIED",
"message", "You do not have permission to access this resource",
"path", request.getServletPath()
));
}
}
```

14.4 Global Exception Handler — @RestControllerAdvice

```
// Centralised error handling for the whole application
@RestControllerAdvice
@Slf4j
public class GlobalExceptionHandler extends ResponseEntityExceptionHandler {

// Security exceptions (re-thrown from service layer)
@ExceptionHandler(AccessDeniedException.class)
public ResponseEntity<ErrorResponse> handleAccessDenied(
AccessDeniedException ex, HttpServletRequest req) {
log.warn("Access denied: {}", req.getRequestURI());
return error(HttpStatus.FORBIDDEN, "ACCESS_DENIED",
"Access denied", req);
}

@ExceptionHandler(BadCredentialsException.class)
public ResponseEntity<ErrorResponse> handleBadCredentials(
BadCredentialsException ex, HttpServletRequest req) {
return error(HttpStatus.UNAUTHORIZED, "INVALID_CREDENTIALS",
"Invalid credentials", req);
}

// Business exceptions
@ExceptionHandler(UserNotFoundException.class)
public ResponseEntity<ErrorResponse> handleUserNotFound(
UserNotFoundException ex, HttpServletRequest req) {
return error(HttpStatus.NOT_FOUND, "USER_NOT_FOUND",
ex.getMessage(), req);
}

@ExceptionHandler(PasswordPolicyViolationException.class)
public ResponseEntity<ErrorResponse> handlePasswordPolicy(
PasswordPolicyViolationException ex, HttpServletRequest req) {
return ResponseEntity.badRequest().body(
ErrorResponse.builder()
.status(400).code("PASSWORD_POLICY_VIOLATION")
.message("Password does not meet policy requirements")
.violations(ex.getViolations())
.path(req.getServletPath())
.timestamp(Instant.now()).build());
}

// Validation errors
@Override
protected ResponseEntity<Object> handleMethodArgumentNotValid(
MethodArgumentNotValidException ex,
HttpHeaders headers, HttpStatusCode status,
WebRequest request) {

List<String> errors = ex.getBindingResult()
.getFieldErrors().stream()
.map(fe -> fe.getField() + ": " + fe.getDefaultMessage())
.collect(Collectors.toList());

return ResponseEntity.badRequest().body(
ErrorResponse.builder()
.status(400).code("VALIDATION_FAILED")
```

```
.message("Request validation failed")
.violations(errors).build());
}

// Catch-all — never expose stack traces
@ExceptionHandler(Exception.class)
public ResponseEntity<ErrorResponse> handleAll(
Exception ex, HttpServletRequest req) {
log.error("Unhandled exception on {}: {}", req.getRequestURI(),
ex.getMessage(), ex);
return error(HttpStatus.INTERNAL_SERVER_ERROR, "INTERNAL_ERROR",
"An unexpected error occurred", req);
}

private ResponseEntity<ErrorResponse> error(HttpStatus status,
String code, String message, HttpServletRequest req) {
return ResponseEntity.status(status).body(
ErrorResponse.builder()
.status(status.value()).code(code).message(message)
.path(req.getServletPath()).timestamp(Instant.now())
.build());
}
}
```

```
// ErrorResponse DTO
@Builder
public record ErrorResponse(
int status,
String code,
String message,
List<String> violations, // null for single errors
String path,
Instant timestamp
) {}
```

14.5 Wire Exception Handlers Into Security Config

```
// Register custom handlers in SecurityFilterChain
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
return http
.exceptionHandling(ex -> ex
// 401 — unauthenticated
.authenticationEntryPoint(customAuthEntryPoint)
// 403 — authenticated but not authorised
.accessDeniedHandler(customAccessDeniedHandler)
)
.build();
}
```

14.6 Exception Handling Quick Reference

Exception	HTTP	Handler	Cause
AuthenticationException	401	AuthenticationEntryPoint	No valid credentials
BadCredentialsException	401	AuthenticationEntryPoint	Wrong password

Exception	HTTP	Handler	Cause
LockedException	401	AuthenticationEntryPoint	Too many failed logins
DisabledException	401	AuthenticationEntryPoint	Account disabled
AccessDeniedException	403	AccessDeniedHandler	Missing role/authority
MethodArgumentNotValid	400	@RestControllerAdvice	Bean validation failed
Exception (catch-all)	500	@RestControllerAdvice	Unexpected error

Chapter 15

Spring Security Filters

15.1 Filter Chain Order

Spring Security's filter chain is an ordered list of Servlet Filters. Key built-in filters and their approximate positions:

Order	Filter	Purpose
100	DisableEncodeUrlFilter	Prevents session ID in URL
200	SecurityContextHolderFilter	Loads SecurityContext per request
300	HeaderWriterFilter	Adds security response headers
400	CorsFilter	Handles CORS preflight
500	CsrfFilter	CSRF token validation
1100	UsernamePasswordAuthFilter	Form login processing
1200	BasicAuthenticationFilter	HTTP Basic auth
1600	ExceptionTranslationFilter	401/403 exception handling
1700	AuthorizationFilter	Final access control decision

15.2 OncePerRequestFilter — Base Class

Always extend **OncePerRequestFilter** for custom security filters. Guarantees **doFilterInternal()** is called exactly once per request, even when RequestDispatcher.forward() is used.

```
// Correct pattern for any custom filter
@Component
public class RequestLoggingFilter extends OncePerRequestFilter {

@Override
protected boolean shouldNotFilter(HttpServletRequest request) {
// Declarative skip logic — return true to bypass this filter
String path = request.getServletPath();
return path.startsWith("/actuator/") || path.endsWith(".css");
}

@Override
protected void doFilterInternal(HttpServletRequest request,
HttpServletResponse response,
FilterChain chain)
throws ServletException, IOException {

long start = System.currentTimeMillis();
MDC.put("requestId", UUID.randomUUID().toString());
response.setHeader("X-Request-Id", MDC.get("requestId"));

try {
chain.doFilter(request, response); // always call this OR write response
log.info("{} {} {}ms status={}",
```

```
request.getMethod(), request.getRequestURI(),
System.currentTimeMillis() - start, response.getStatus());
} finally {
MDC.clear(); // CRITICAL — always clear MDC
}
}
}
```

15.3 JWT Authentication Filter — Production Grade

```
// Complete JWT filter with all edge cases
@Component @RequiredArgsConstructor
public class JwtAuthenticationFilter extends OncePerRequestFilter {

private static final Set&lt;String&gt; SKIP = Set.of(
"/api/auth/login", "/api/auth/register",
"/api/auth/refresh", "/actuator/health");

@Override
protected boolean shouldNotFilter(HttpServletRequest req) {
return SKIP.contains(req.getServletPath()) ||
HttpMethod.OPTIONS.name().equals(req.getMethod());
}

@Override
protected void doFilterInternal(HttpServletRequest req,
HttpServletResponse res,
FilterChain chain)
throws ServletException, IOException {

String token = extractToken(req);
if (!StringUtils.hasText(token)) {
chain.doFilter(req, res); return;
}

try {
Claims claims = jwtService.validateAndExtractClaims(token);

if (!"ACCESS".equals(claims.get("type")))
throw new InvalidTokenException("Not an access token");

if (blacklistService.isBlacklisted(claims.getId()))
throw new InvalidTokenException("Token revoked");

// Build principal from claims — ZERO DB reads
CustomUserPrincipal principal = CustomUserPrincipal.builder()
.userId(Long.valueOf(claims.getSubject()))
.email(claims.get("email", String.class))
.tenantId(claims.get("tenantId", String.class))
.build();

SecurityContextHolder.getContext().setAuthentication(
new UsernamePasswordAuthenticationToken(
principal, null,
jwtService.extractAuthorities(token)));

MDC.put("userId", principal.getUserId().toString());
```

```
MDC.put("tenantId", principal.getTenantId());

chain.doFilter(req, res);

} catch (TokenExpiredException ex) {
sendError(res, 401, "TOKEN_EXPIRED", ex.getMessage());
} catch (InvalidTokenException ex) {
sendError(res, 401, "INVALID_TOKEN", ex.getMessage());
}
}

private String extractToken(HttpServletRequest req) {
String header = req.getHeader(HttpHeaders.AUTHORIZATION);
return StringUtils.hasText(header) &amp;&amp; header.startsWith("Bearer ")
? header.substring(7).trim() : null;
}

private void sendError(HttpServletResponse res,
int status, String code, String msg)
throws IOException {
SecurityContextHolder.clearContext();
res.setStatus(status);
res.setContentType(MediaType.APPLICATION_JSON_VALUE);
res.getWriter().write(
"{"status":%d,"code":"%s","message":"%s"}"
.formatted(status, code, msg));
}
}
```

15.4 Filter Placement Guide

Placement	Method	Use for
Before SecurityContextHolderFilter	addFilterBefore(f, SecurityContextHolderFilter.class)	Request validation, IP blocking
Before UsernamePasswordFilter	addFilterBefore(f, UsernamePasswordAuthFilter.class)	JWT auth, API key auth
After UsernamePasswordFilter	addFilterAfter(f, UsernamePasswordAuthFilter.class)	Tenant context, MFA check
After AuthorizationFilter	addFilterAfter(f, AuthorizationFilter.class)	Response headers, audit logging

15.5 Custom Filter Gallery — Summary

Filter	Position	Purpose
RequestValidationFilter	Before SecurityContext	URI length, path traversal, blocked headers
RequestLoggingFilter	After Validation	MDC enrichment, request/response timing
ApiKeyAuthFilter	Before UsernamePassword	X-Api-Key header authentication
JwtAuthenticationFilter	Before UsernamePassword	JWT validation + SecurityContext population
TenantContextFilter	After JWT	Tenant resolution + ThreadLocal storage

Filter	Position	Purpose
SecurityHeadersFilter	After Authorization	CSP, HSTS, X-Frame-Options on responses

15.6 OncePerRequestFilter Contract

- Override shouldNotFilter() for declarative skip logic — cleaner than if-blocks inside doFilterInternal
- Always either call chain.doFilter(request, response) OR write directly to response — never both
- Always clear ThreadLocal state (MDC, TenantContext) in a finally block — thread pool reuse
- Never throw unchecked exceptions from doFilterInternal — catch them and write an error response
- Annotate with @Component and register via http.addFilterBefore() in SecurityConfig

Chapter 16

Database Security

16.1 SQL Injection — The Root Cause

SQL Injection remains the most dangerous web vulnerability (OWASP #1). It occurs when user-supplied input is concatenated directly into SQL strings, allowing attackers to alter query logic, bypass authentication, exfiltrate data, or destroy entire databases. The fix is always the same: **never build SQL by concatenating user input**.

```
// VULNERABLE vs SAFE — always use parameterized queries
// ATTACK: username = admin' OR '1'='1
// Query becomes: SELECT * FROM users WHERE username='admin' OR '1'='1'
// Result: returns ALL users, bypassing login

// WRONG — string concatenation — NEVER do this
String sql = "SELECT * FROM users WHERE username='" + username + "'";
Statement st = conn.createStatement();
ResultSet rs = st.executeQuery(sql); // SQL injection possible!

// CORRECT — PreparedStatement with bound parameters
String sql = "SELECT * FROM users WHERE username = ?";
PreparedStatement ps = conn.prepareStatement(sql);
ps.setString(1, username); // username is DATA, not code
ResultSet rs = ps.executeQuery(); // safe regardless of input content

// CORRECT — NamedParameterJdbcTemplate (Spring)
String sql = "SELECT * FROM users WHERE username = :username AND active = :active";
MapSqlParameterSource params = new MapSqlParameterSource()
.addValue("username", username)
.addValue("active", true);
User user = namedJdbc.queryForObject(sql, params, userRowMapper);
```

16.2 Spring Data JPA — Safe Query Patterns

Spring Data JPA generates safe parameterized SQL for all derived query methods and JPQL queries using **@Param**. Always use method derivation or JPQL — never native queries with concatenation.

```
// All safe patterns in one repository
@Repository
public interface UserRepository extends JpaRepository<User, Long> {

// Pattern 1: Derived query — Spring generates safe parameterized SQL
Optional<User> findByEmailAndEnabled(String email, boolean enabled);

// Pattern 2: JPQL with @Param — always use named parameters
@Query("SELECT u FROM User u WHERE u.tenantId = :tenantId " +
"AND u.role.name = :role ORDER BY u.createdAt DESC")
List<User> findByTenantAndRole(@Param("tenantId") String tenantId,
@Param("role") String role);

// Pattern 3: Native SQL — still use :param, never concatenate
@Query(value = "SELECT * FROM users WHERE tenant_id = :tenantId " +
"AND created_at > :since",
nativeQuery = true)
```

```
List<User> findRecentByTenant(@Param("tenantId") String tenantId,
@Param("since") Instant since);

// Pattern 4: Specifications — dynamic queries without injection risk
// (see 16.3 below)

// WRONG — NEVER do this even in JPQL
// @Query("SELECT u FROM User u WHERE u.username = '" + username + "'")
}
```

// JPA Specifications — safe dynamic queries

```
// Instead of building query strings dynamically, use Specification<T>
public class UserSpecifications {

public static Specification<User> hasEmail(String email) {
return (root, query, cb) ->
email == null ? cb.conjunction()
: cb.equal(root.get("email"), email);
}

public static Specification<User> inTenant(String tenantId) {
return (root, query, cb) ->
tenantId == null ? cb.conjunction()
: cb.equal(root.get("tenantId"), tenantId);
}

public static Specification<User> createdAfter(Instant since) {
return (root, query, cb) ->
since == null ? cb.conjunction()
: cb.greaterThan(root.get("createdAt"), since);
}

public static Specification<User> searchByName(String term) {
return (root, query, cb) ->
!StringUtils.hasText(term) ? cb.conjunction()
: cb.or(
cb.like(cb.lower(root.get("firstName")),
"%" + term.toLowerCase() + "%"),
cb.like(cb.lower(root.get("lastName")),
"%" + term.toLowerCase() + "%")
);
// cb.like with user input is safe — it is parameterized internally
// The % wildcards are added server-side, not from user input
}
}

// Usage — build dynamic filter safely
@Service
public class UserSearchService {

public Page<User> search(UserSearchRequest req, Pageable page) {
Specification<User> spec = Specification
.where(inTenant(req.getTenantId()))
.and(hasEmail(req.getEmail()))
.and(searchByName(req.getName()))
.and(createdAfter(req.getSince()));

return userRepository.findAll(spec, page);
```

```
}
}
```

16.3 Row-Level Security — Tenant Isolation

In multi-tenant applications every query must be scoped to the current tenant. The safest pattern is to enforce tenant filtering at the repository layer so it is impossible to accidentally query another tenant's data.

```
// Approach 1: @Filter on entity — Hibernate-level automatic filter
@Entity
@Table(name = "orders")
@FilterDef(name = "tenantFilter",
parameters = @ParamDef(name = "tenantId", type = String.class))
@Filter(name = "tenantFilter", condition = "tenant_id = :tenantId")
public class Order {
@Id @GeneratedValue private Long id;
@Column(nullable = false) private String tenantId;
// ... other fields
}

// Enable filter per-session in a filter/interceptor
@Component
@RequiredArgsConstructor
public class TenantFilterActivator extends OncePerRequestFilter {

private final EntityManager entityManager;

@Override
protected void doFilterInternal(HttpServletRequest req,
HttpServletResponse res,
FilterChain chain)
throws ServletException, IOException {
String tenantId = TenantContext.getTenantId();
if (tenantId != null) {
Session session = entityManager.unwrap(Session.class);
session.enableFilter("tenantFilter")
.setParameter("tenantId", tenantId);
}
try { chain.doFilter(req, res); }
finally {
Session session = entityManager.unwrap(Session.class);
session.disableFilter("tenantFilter");
}
}
}
```

```
// Approach 2: All repository methods scoped by tenantId (explicit)
@Repository
public interface OrderRepository extends JpaRepository<Order, Long> {

// Every query includes tenantId — impossible to leak cross-tenant
Page<Order> findByTenantId(String tenantId, Pageable pageable);

Optional<Order> findByIdAndTenantId(Long id, String tenantId);

@Query("SELECT o FROM Order o WHERE o.tenantId = :tenantId " +
"AND o.status = :status")
```

```
List<Order> findByTenantAndStatus(
@Param("tenantId") String tenantId,
@Param("status") OrderStatus status);

@Modifying
@Query("DELETE FROM Order o WHERE o.id = :id AND o.tenantId = :tenantId")
int deleteByIdAndTenantId(
@Param("id") Long id,
@Param("tenantId") String tenantId);
// Returns 0 if order belongs to different tenant — safe no-op
}
```

```
// Approach 3: Base entity with @PrePersist tenant enforcement
@MappedSuperclass
public abstract class TenantAwareEntity {

@Column(nullable = false, updatable = false)
private String tenantId;

@PrePersist
public void prePersist() {
// Auto-set tenantId from SecurityContext on every insert
if (this.tenantId == null) {
Authentication auth = SecurityContextHolder.getContext()
.getAuthentication();
if (auth != null &&
auth.getPrincipal() instanceof CustomUserPrincipal p) {
this.tenantId = p.getTenantId();
}
}
}
}

@Entity
public class Order extends TenantAwareEntity {
// tenantId is set automatically on save — no manual code needed
}
```

16.4 Secure Password Storage

Passwords must never be stored in plain text, reversible encryption, or fast hashes (MD5, SHA-1, SHA-256). Always use a slow, salted, adaptive hashing algorithm. Spring Security's BCrypt with strength 12 is the production standard.

```
// Schema — password column must be long enough for hash
-- BCrypt hash is always 60 chars; allow 255 for future algorithm upgrades
ALTER TABLE users ALTER COLUMN password TYPE VARCHAR(255);

-- DelegatingPasswordEncoder stores: {bcrypt}$2a$12$<hash>
-- Format includes algorithm prefix so you can migrate algorithms
```

```
// Registration — hash before any DB write
@Service @RequiredArgsConstructor @Transactional
public class UserService {

private final PasswordEncoder passwordEncoder; // BCryptPasswordEncoder(12)
private final UserRepository userRepository;
```

```
public User createUser(CreateUserRequest req) {
// ALWAYS encode before save — never store raw password
String hashedPassword = passwordEncoder.encode(req.getPassword());
req.clearPassword(); // zero out raw password from memory ASAP

return userRepository.save(User.builder()
.email(req.getEmail())
.password(hashedPassword)
.build());
}

// Password reset — hash the new password
@Transactional
public void resetPassword(Long userId, String newPassword) {
User user = userRepository.findById(userId).orElseThrow();
user.setPassword(passwordEncoder.encode(newPassword));
user.setPasswordChangedAt(Instant.now());
user.setCredentialsExpired(false);
userRepository.save(user);
}
}
```

```
// Never log, return, or expose passwords
// WRONG — exposes password in logs
log.info("User {} registered with password {}", email, password);

// WRONG — returns password hash in API response
return new UserDto(user.getId(), user.getEmail(), user.getPassword());

// CORRECT — @JsonIgnore on the password field
@JsonIgnore
private String password;

// CORRECT — DTO never includes password
public record UserResponseDto(Long id, String email, String username) {
public static UserResponseDto from(User u) {
return new UserResponseDto(u.getId(), u.getEmail(), u.getUsername());
// password intentionally excluded
}
}
```

16.5 Database Audit Logging

Every data modification must be traceable — who changed what, when, from where. Spring Data's auditing hooks provide this automatically.

```
// JPA Auditing — auto-populate created/modified metadata
// Enable in config
@Configuration
@EnableJpaAuditing(auditorAwareRef = "auditorProvider")
public class JpaAuditConfig {

@Bean
public AuditorAware<String> auditorProvider() {
return () -> Optional.ofNullable(
SecurityContextHolder.getContext().getAuthentication())
.filter(Authentication::isAuthenticated)
```

```
.map(auth -> {
if (auth.getPrincipal() instanceof CustomUserPrincipal p)
return p.getEmail();
return auth.getName();
});
}
}

// Audit base entity — extend all auditable entities from this
@MappedSuperclass
@EntityListeners(AuditingEntityListener.class)
public abstract class AuditableEntity {

@CreatedBy @Column(updatable = false)
private String createdBy; // email of user who created the record

@LastModifiedBy
private String lastModifiedBy; // email of user who last changed it

@CreatedDate @Column(updatable = false)
private Instant createdAt;

@LastModifiedDate
private Instant updatedAt;
}
```

// Custom audit log table — full change history

```
@Entity
@Table(name = "audit_log")
@Data @Builder @NoArgsConstructor @AllArgsConstructor
public class AuditLog {
@Id @GeneratedValue private Long id;

@Column(nullable = false) private String entityType; // "Order", "User"
@Column(nullable = false) private Long entityId;
@Column(nullable = false) private String action; // CREATE UPDATE DELETE

@Column(columnDefinition = "TEXT") private String oldValue; // JSON
@Column(columnDefinition = "TEXT") private String newValue; // JSON

private String performedBy; // email
private String tenantId;
private String ipAddress;
private String traceId;

@Column(nullable = false) private Instant performedAt;
}

// AuditService — publish audit events
@Service @RequiredArgsConstructor
public class AuditService {

private final AuditLogRepository auditRepository;
private final ObjectMapper objectMapper;

@Async // fire-and-forget — never block main request thread
public void logChange(String entityType, Long entityId,
String action, Object oldVal, Object newVal) {
```

```
Authentication auth = SecurityContextHolder.getContext()
.getAuthentication();
String user = auth != null ? auth.getName() : "system";
String tid = auth != null &&
auth.getPrincipal() instanceof CustomUserPrincipal p
? p.getTenantId() : null;

auditRepository.save(AuditLog.builder()
.entityType(entityType).entityId(entityId).action(action)
.oldValue(toJson(oldVal)).newValue(toJson(newVal))
.performedBy(user).tenantId(tid)
.traceId(MDC.get("traceId"))
.performedAt(Instant.now())
.build());
}

private String toJson(Object obj) {
if (obj == null) return null;
try { return objectMapper.writeValueAsString(obj); }
catch (Exception e) { return obj.toString(); }
}
}
```

```
// Hibernate Envers — automatic entity history (alternative)
// Add @Audited to any entity — Envers records every change automatically
@Entity
@Audited
public class Order extends AuditableEntity {
@Id @GeneratedValue private Long id;
private String status;
private BigDecimal total;
// Envers creates orders_AUD table automatically
}

// Query audit history
@Service @RequiredArgsConstructor
public class OrderHistoryService {

private final AuditReader auditReader;

public List<Order> getFullHistory(Long orderId) {
return auditReader.createQuery()
.forRevisionsOfEntity(Order.class, true, true)
.add(AuditEntity.id().eq(orderId))
.addOrder(AuditEntity.revisionNumber().asc())
.getResultList();
}
}
```

16.6 Connection Security — SSL/TLS

```
// application.yml — enforce SSL for DB connections
spring:
datasource:
url: jdbc:postgresql://db.yourapp.com:5432/appdb?ssl=true&sslmode=verify-full
username: ${DB_USER}
```

```
password: ${DB_PASSWORD}
hikari:
# Connection pool security settings
maximum-pool-size: 20
minimum-idle: 5
connection-timeout: 30000
idle-timeout: 600000
max-lifetime: 1800000
# Verify connection health
connection-test-query: SELECT 1
data-source-properties:
ssl: true
sslmode: verify-full
sslrootcert: /certs/db-ca.crt # CA cert for server verification
```

```
// Least-privilege DB user — one user per service
-- Create service-specific DB user with minimum required privileges
-- Order service only needs access to orders schema
CREATE USER order_service_user WITH PASSWORD '${STRONG_RANDOM_PASSWORD}';

GRANT CONNECT ON DATABASE appdb TO order_service_user;
GRANT USAGE ON SCHEMA orders TO order_service_user;

-- Only the tables this service needs
GRANT SELECT, INSERT, UPDATE ON orders.orders TO order_service_user;
GRANT SELECT, INSERT ON orders.order_items TO order_service_user;
GRANT SELECT ON orders.products TO order_service_user;

-- NEVER grant DELETE or DROP in production (use soft deletes instead)
-- NEVER use the postgres superuser for application connections

-- Separate read-only user for reporting/analytics
CREATE USER reporting_user WITH PASSWORD '${REPORTING_PASSWORD}';
GRANT CONNECT ON DATABASE appdb TO reporting_user;
GRANT USAGE ON SCHEMA orders TO reporting_user;
GRANT SELECT ON ALL TABLES IN SCHEMA orders TO reporting_user;
```

16.7 Input Sanitization — Defence in Depth

```
// Sanitize before storage — belt and suspenders
@Service
@RequiredArgsConstructor
public class InputSanitizationService {

// OWASP Java HTML Sanitizer
private static final PolicyFactory HTML_POLICY =
Sanitizers.FORMATTING.and(Sanitizers.LINKS);

// Strip any HTML from free-text fields stored in DB
public String sanitizeHtml(String input) {
if (!StringUtils.hasText(input)) return input;
return HTML_POLICY.sanitize(input);
}

// Remove null bytes — can cause issues in some DB drivers
public String removeNullBytes(String input) {
return input == null ? null : input.replace("■", "");
```

```
}

// Validate enum-like string params rather than trusting them
public OrderStatus parseOrderStatus(String raw) {
try { return OrderStatus.valueOf(raw.toUpperCase()); }
catch (IllegalArgumentException e) {
throw new BadRequestException("Invalid status: " + raw);
}
}

// Strip leading/trailing whitespace — normalize before storage
public String normalize(String input) {
return input == null ? null
: removeNullBytes(input.trim());
}
}

// Wire into service layer
@Service @RequiredArgsConstructor
public class ProductService {

private final InputSanitizationService sanitizer;

public ProductDto createProduct(CreateProductRequest req) {
Product product = Product.builder()
// Sanitize all free-text fields before persistence
.name(sanitizer.normalize(req.getName()))
.description(sanitizer.sanitizeHtml(req.getDescription()))
.sku(sanitizer.normalize(req.getSku()))
.build();
return ProductDto.from(productRepository.save(product));
}
}
```

16.8 Quick Reference — Database Security Checklist

Area	Rule	Implementation
SQL Injection	Never concatenate user input into SQL	Spring Data JPA, @Param, PreparedStatement
Dynamic Queries	Use Specification for dynamic filters	JpaSpecificationExecutor + Specification.where()
Tenant Isolation	Every query scoped to tenantId	@Filter, repo methods with tenantId param, @PrePersist
Passwords	Slow adaptive hash, never plain text	BCryptPasswordEncoder(12), DelegatingPasswordEncoder
Audit Trail	Record all writes with who/when/what	Spring Data Auditing, AuditLog entity, Hibernate Envers
Transport	Encrypt DB connections in transit	JDBC SSL url, sslmode=verify-full, CA cert validation
Credentials	Least-privilege per-service DB users	GRANT only needed tables; never superuser in production
Input	Sanitize free-text before storage	OWASP Java HTML Sanitizer, null-byte removal, normalize
Logging	Never log passwords or sensitive data	@JsonIgnore on password, DTO excludes sensitive fields

16.9 Testing — Database Security

```
// Verify SQL injection protection and tenant isolation
@DataJpaTest
@Import(UserRepository.class)
class DatabaseSecurityTest {

@Autowired UserRepository userRepository;
@Autowired TestEntityManager em;

@Test
void sqlInjection_inDerivedQuery_isSafe() {
// Even a crafted injection string is treated as literal data
String injected = "admin' OR '1'='1";
Optional<User> result = userRepository.findByEmail(injected);
// Returns empty — injection string finds no real user
assertThat(result).isEmpty();
}

@Test
void tenantIsolation_cannotReadOtherTenantData() {
// Seed two tenants
User tenantA = em.persist(User.builder()
.email("a@a.com").tenantId("tenant-a").build());
User tenantB = em.persist(User.builder()
.email("b@b.com").tenantId("tenant-b").build());
em.flush();

// Query scoped to tenant-a must NOT return tenant-b user
List<User> results = userRepository.findByTenantId("tenant-a",
PageRequest.of(0, 100)).getContent();

assertThat(results).extracting(User::getTenantId)
.containsOnly("tenant-a")
.doesNotContain("tenant-b");
}

@Test
void passwordStorage_isHashedNotPlainText() {
User user = em.persist(User.builder()
.email("test@test.com")
.password(new BCryptPasswordEncoder(12).encode("P@ssword1!"))
.build());
em.flush(); em.clear();

User loaded = userRepository.findById(user.getId()).orElseThrow();
// Stored value must start with BCrypt prefix — never plain text
assertThat(loaded.getPassword()).startsWith("$2a$");
assertThat(loaded.getPassword()).doesNotContain("P@ssword1!");
}

@Test
void auditLog_capturesChange() {
// Perform a write operation
User user = createTestUser();
user.setEmail("newemail@test.com");
userRepository.save(user);
```

```
// Verify audit entry was created
List<AuditLog> logs = auditLogRepository
.findByEntityTypeAndEntityId("User", user.getId());
assertThat(logs).isNotEmpty();
assertThat(logs.get(0).getAction()).isEqualTo("UPDATE");
assertThat(logs.get(0).getPerformedBy()).isNotBlank();
}
}
```

Chapter 17

HTTPS & SSL / TLS

17.1 SSL/TLS Basics

TLS (Transport Layer Security) encrypts all data between client and server, preventing eavesdropping and man-in-the-middle attacks. HTTPS = HTTP over TLS. Every production Spring Boot application must run on HTTPS. Never transmit credentials, tokens, or sensitive data over plain HTTP.

Protocol	Status	Notes
SSL 2.0 / 3.0	Broken	Never use — POODLE, DROWN attacks
TLS 1.0 / 1.1	Deprecated	Disable — PCI-DSS non-compliant since 2022
TLS 1.2	Acceptable	Minimum for production — use strong ciphers
TLS 1.3	Recommended	Fastest handshake, best security, forward secrecy

17.2 Generate Keystore

```
# Create self-signed keystore for dev (use CA cert in prod)
# Generate RSA-2048 keypair in PKCS12 keystore
keytool -genkeypair \
-alias spring-app \
-keyalg RSA \
-keysize 2048 \
-storetype PKCS12 \
-keystore keystore.p12 \
-validity 3650 \
-storepass changeit \
-dname "CN=localhost, OU=Dev, O=YourCompany, C=US"

# Production: use Let's Encrypt or your CA
# certbot certonly --standalone -d yourapp.com
# Convert to PKCS12:
# openssl pkcs12 -export -in fullchain.pem -inkey privkey.pem \
# -out keystore.p12 -name spring-app -password pass:yourpassword
```

17.3 Enable HTTPS in Spring Boot

```
# application.yml — full HTTPS config
server:
port: 8443
ssl:
enabled: true
key-store: classpath:keystore.p12 # or file:/etc/ssl/keystore.p12
key-store-password: ${SSL_KEYSTORE_PASSWORD}
key-store-type: PKCS12
key-alias: spring-app
# TLS version — enforce 1.2+
protocol: TLS
enabled-protocols:
- TLSv1.2
```

```
- TLSv1.3
# Strong cipher suites only
ciphers:
- TLS_AES_128_GCM_SHA256 # TLS 1.3
- TLS_AES_256_GCM_SHA384 # TLS 1.3
- TLS_CHACHA20_POLY1305_SHA256 # TLS 1.3
- TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384 # TLS 1.2
- TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256 # TLS 1.2
```

17.4 Redirect HTTP to HTTPS

```
// Force all HTTP traffic to HTTPS
```

```
// Run two connectors: 8080 (HTTP, redirect only) + 8443 (HTTPS)
@Configuration
public class HttpsRedirectConfig {

@Bean
public ServletWebServerFactory servletContainer() {
TomcatServletWebServerFactory tomcat =
new TomcatServletWebServerFactory() {
@Override
protected void postProcessContext(Context context) {
SecurityConstraint sc = new SecurityConstraint();
sc.setUserConstraint("CONFIDENTIAL"); // requires HTTPS
SecurityCollection collection = new SecurityCollection();
collection.addPattern("/*"); // all paths
sc.addCollection(collection);
context.addConstraint(sc);
}
};

// Add HTTP connector on port 8080 that redirects to 8443
tomcat.addAdditionalTomcatConnectors(httpRedirectConnector());
return tomcat;
}

private Connector httpRedirectConnector() {
Connector connector =
new Connector(TomcatServletWebServerFactory.DEFAULT_PROTOCOL);
connector.setScheme("http");
connector.setPort(8080);
connector.setSecure(false);
connector.setRedirectPort(8443);
return connector;
}
}
```

17.5 HSTS — HTTP Strict Transport Security

```
// Tell browsers: always use HTTPS for this domain
```

```
http.headers(headers -> headers
.httpStrictTransportSecurity(hsts -> hsts
.maxAgeInSeconds(31536000) // 1 year — browsers cache this
.includeSubDomains(true) // all subdomains too
.preload(true) // submit to browser preload lists
)
```

```
);

// Resulting header on every response:
// Strict-Transport-Security: max-age=31536000; includeSubDomains; preload

// After first visit, browser will REFUSE to connect via HTTP
// even if user types http:// in address bar
```

17.6 Certificate Pinning (Mobile/API)

```
// Pin expected certificate fingerprint in client
```

```
// Android — pin in network_security_config.xml
<network-security-config>
<domain-config>
<domain includeSubdomains="true">api.yourapp.com</domain>
<pin-set expiration="2025-12-31">
<!-- SHA-256 fingerprint of your cert's public key -->
<pin digest="SHA-256">base64EncodedSHA256OfPublicKey==</pin>
<!-- Backup pin (different key) to avoid lockout -->
<pin digest="SHA-256">backupPinBase64==</pin>
</pin-set>
</domain-config>
</network-security-config>

// Java (OkHttp)
OkHttpClient client = new OkHttpClient.Builder()
.certificatePinner(new CertificatePinner.Builder()
.add("api.yourapp.com",
"sha256/YourCertificatePublicKeyHash=")
.build())
.build();
```

17.7 HTTPS Checklist

- Enable TLS 1.2 and 1.3 only — disable SSL 3.0, TLS 1.0, TLS 1.1
- Use 2048-bit RSA or 256-bit ECDSA certificates minimum
- Enable HSTS with max-age=31536000, includeSubDomains
- Redirect all HTTP requests to HTTPS (301 permanent)
- Use strong cipher suites — prefer ECDHE (forward secrecy)
- Renew certificates before expiry (Let's Encrypt auto-renews every 90 days)
- Never log SSL passwords or private key material
- Store keystore password in environment variable, not in application.yml

Chapter 18

Security Headers

18.1 Spring Security Default Headers

Spring Security adds a set of security response headers by default. These instruct the browser to apply additional protections. You can customise or extend them in your SecurityFilterChain.

```
// Default headers Spring Security adds automatically
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
X-XSS-Protection: 0
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Pragma: no-cache
Expires: 0
Strict-Transport-Security: max-age=31536000; includeSubDomains (HTTPS only)
```

18.2 Content Security Policy (CSP)

CSP is the most powerful defence against XSS. It tells the browser exactly which sources are trusted for scripts, styles, images, and other resources.

```
// CSP configuration
http.headers(headers -> headers
.contentSecurityPolicy(csp -> csp.policyDirectives(
// Only load content from your own domain
"default-src 'self'; " +

// Scripts — only self + nonce for inline
"script-src 'self' 'nonce-{nonce}'; " +

// Styles
"style-src 'self' https://fonts.googleapis.com; " +

// Fonts
"font-src 'self' https://fonts.gstatic.com; " +

// Images — self + data URIs + HTTPS
"img-src 'self' data: https:; " +

// API calls — only your domains
"connect-src 'self' https://api.yourapp.com; " +

// Block all plugins
"object-src 'none'; " +

// Disallow embedding in iframes
"frame-ancestors 'none'; " +

// Force HTTPS for all resources
"upgrade-insecure-requests; " +

// Violation reporting
"report-uri /api/csp-report"
))
```

```
);
```

```
// CSP violation endpoint — monitor and tune your policy
@RestController
@RequestMapping("/api")
public class CspReportController {

@PostMapping(value = "/csp-report",
consumes = "application/csp-report")
@ResponseStatus(HttpStatus.NO_CONTENT)
public void receiveCspReport(
@RequestBody CspViolationReport report) {
log.warn("CSP violation: blockedUri={} violatedDirective={}",
report.getBlockedUri(), report.getViolatedDirective());
// Store in monitoring system / alert if spike
}
}
```

18.3 All Security Headers Reference

Header	What it does	Recommended value
X-Content-Type-Options	Prevents MIME sniffing	nosniff
X-Frame-Options	Prevents clickjacking	DENY or SAMEORIGIN
X-XSS-Protection	Legacy XSS filter (modern: use CSP)	0 (disabled — use CSP instead)
Strict-Transport-Security	Forces HTTPS	max-age=31536000; includeSubDomains
Content-Security-Policy	Controls resource loading	default-src 'self'; ...
Referrer-Policy	Controls Referer header leakage	strict-origin-when-cross-origin
Permissions-Policy	Disables browser features	camera=(), microphone=(), geolocation=()
Cache-Control	Prevents sensitive data caching	no-store, no-cache (for API responses)

18.4 Full Headers Configuration

```
// Complete header setup
http.headers(headers -> headers
// Prevent MIME sniffing
.contentTypeOptions(Customizer.withDefaults())

// Clickjacking — deny all framing
.frameOptions(fo -> fo.deny())

// HSTS
.httpStrictTransportSecurity(hsts -> hsts
.maxAgeInSeconds(31536000)
.includeSubDomains(true)
.preload(true))

// CSP
.contentSecurityPolicy(csp -> csp
.policyDirectives("default-src 'self'; script-src 'self'; " +
"object-src 'none'; frame-ancestors 'none'"))
```

```
// Referrer Policy
.referrerPolicy(rp -> rp
.policy(ReferrerPolicyHeaderWriter.ReferrerPolicy
.STRICT_ORIGIN_WHEN_CROSS_ORIGIN))

// Permissions Policy
.permissionsPolicy(pp -> pp
.policy("camera=(), microphone=(), geolocation=(), payment=()"))

// Cache control for all responses
.cacheControl(Customizer.withDefaults())
);
```

18.5 Permissions-Policy — Disable Sensitive Browser APIs

```
// Lock down what the browser can do on your pages
```

```
Permissions-Policy: camera=(),
microphone=(),
geolocation=(),
payment=(),
usb=(),
fullscreen=(self),
display-capture=()

// Prevents malicious scripts from accessing camera/mic even if XSS occurs
```

Chapter 19

Method-Level Security (Advanced)

19.1 Enable Method Security

```
// Full method security enablement
@Configuration
@EnableMethodSecurity(
prePostEnabled = true, // @PreAuthorize, @PostAuthorize, @Pre/PostFilter
securedEnabled = true, // @Secured
jsr250Enabled = true // @RolesAllowed, @PermitAll, @DenyAll
)
public class MethodSecurityConfig {

// Custom method security expression handler
@Bean
public MethodSecurityExpressionHandler methodSecurityExpressionHandler(
RoleHierarchy roleHierarchy) {
DefaultMethodSecurityExpressionHandler handler =
new DefaultMethodSecurityExpressionHandler();
handler.setRoleHierarchy(roleHierarchy);
handler.setDefaultRolePrefix("ROLE_");
return handler;
}
}
```

19.2 Expression-Based Access Control — SpEL

```
// Rich SpEL expressions in @PreAuthorize
// 1. Role + authority combined
@PreAuthorize("hasRole('ADMIN') and hasAuthority('report:export')")
public byte[] exportReport(Long reportId) { ... }

// 2. Method parameter in expression
@PreAuthorize("authentication.principal.tenantId == #tenantId")
public List<OrderDto> getOrdersForTenant(String tenantId) { ... }

// 3. Custom bean method
@PreAuthorize("@securityService.canAccessOrder(#orderId, authentication)")
public OrderDto getOrder(Long orderId) { ... }

// 4. SpEL with null-safe navigation
@PreAuthorize("authentication.principal?.userId != null")
public UserDto getProfile() { ... }

// 5. Collection check
@PreAuthorize("#requestedScopes.containsAll(authentication.principal.scopes)")
public ApiResponse query(Set<String> requestedScopes) { ... }

// 6. Environment variable check (time-based)
@PreAuthorize("@businessHours.isOpen() or hasRole('ADMIN')")
public TradingDto placeTrade(TradeRequest req) { ... }
```

19.3 Custom Security Expressions

```
// Extend Spring's security expression root
// Custom expression root — adds methods available in SpEL
public class CustomSecurityExpressionRoot
extends SecurityExpressionRoot
implements MethodSecurityExpressionOperations {

private final TenantService tenantService;
private Object filterObject, returnObject;

public CustomSecurityExpressionRoot(Authentication auth,
TenantService tenantService) {
super(auth);
this.tenantService = tenantService;
}

// Usage: @PreAuthorize("isTenantAdmin()")
public boolean isTenantAdmin() {
if (getPrincipal() instanceof CustomUserPrincipal p) {
return tenantService.isAdmin(p.getTenantId(), p.getUserId());
}
return false;
}

// Usage: @PreAuthorize("hasPlan('ENTERPRISE')")
public boolean hasPlan(String plan) {
if (getPrincipal() instanceof CustomUserPrincipal p) {
return tenantService.getTenantPlan(p.getTenantId())
.equals(plan);
}
return false;
}

@Override public void setFilterObject(Object o) { filterObject = o; }
@Override public Object getFilterObject() { return filterObject; }
@Override public void setReturnObject(Object o) { returnObject = o; }
@Override public Object getReturnObject() { return returnObject; }
@Override public Object getThis() { return this; }
}
```

19.4 Securing Service Layer vs Controller Layer

Always secure the **service layer**, not just controllers. If you only secure controllers, internal service-to-service calls bypass security. Dual-layer security (controller + service) provides defence in depth.

```
// Service layer security — always do this
@Service
public class PaymentService {

// Controller security is a convenience — service security is mandatory
@PreAuthorize("hasAuthority('payment:write') and " +
"@paymentSecurity.belongsToUser(#req.accountId, authentication)")
@Transactional
public PaymentDto processPayment(PaymentRequest req) {
// Even if called internally from another service,
// @PreAuthorize is evaluated via Spring AOP proxy
// (as long as the call goes through the proxy, not this.method())
```

```
return execute(req);
}
}

// IMPORTANT: @PreAuthorize is bypassed if called via this.method()
// Always inject self or use AspectJ weaving for internal calls
@Service
@RequiredArgsConstructor
public class OrderService {
private final PaymentService paymentService; // injected proxy — SAFE

public void placeOrder(OrderRequest req) {
// This goes through Spring proxy → @PreAuthorize enforced
paymentService.processPayment(req.toPaymentRequest());
}
}
```

19.5 Method Security Checklist

Annotation	When evaluated	Access to return value	Best for
@PreAuthorize	Before method	No	Role/permission/ownership gates
@PostAuthorize	After method	Yes	Ownership check on loaded object
@PreFilter	Before method	No	Filter input collection
@PostFilter	After method	Yes	Filter output collection
@Secured	Before method	No	Simple role list, no SpEL
@RolesAllowed	Before method	No	JSR-250 standard, no ROLE_ prefix
@PermitAll	Before method	No	Explicitly open a method
@DenyAll	Before method	No	Block all access to a method

Chapter 20

LDAP Authentication

20.1 LDAP Basics

LDAP (Lightweight Directory Access Protocol) is used in enterprises to centralise user identity management — particularly Microsoft Active Directory (AD). Spring Security supports both embedded LDAP (for testing) and remote LDAP/AD servers (for production).

Term	Meaning	Example
DN	Distinguished Name — unique path to an entry	cn=Alice,ou=Users,dc=yourcompany,dc=com
DC	Domain Component	dc=yourcompany,dc=com
CN	Common Name	cn=Alice Smith
OU	Organisational Unit	ou=Engineering
UID	User ID attribute	uid=alice
GID	Group ID / Group membership	cn=Developers,ou=Groups

20.2 Dependencies & Config

```
<dependency>
<groupId>org.springframework.security</groupId>
<artifactId>spring-security-ldap</artifactId>
</dependency>
<dependency>
<groupId>com.unboundid</groupId>
<artifactId>unboundid-ldapsdk</artifactId> <!-- embedded LDAP for tests -->
<scope>test</scope>
</dependency>
```

```
# application.yml
```

```
spring:
ldap:
urls: ldap://ldap.yourcompany.com:389
base: dc=yourcompany,dc=com
username: cn=SpringApp,ou=ServiceAccounts,dc=yourcompany,dc=com
password: ${LDAP_BIND_PASSWORD}

app:
ldap:
user-search-base: ou=Users
user-search-filter: (uid={0}) # {0} = supplied username
group-search-base: ou=Groups
group-search-filter: (member={0})
```

20.3 LDAP Authentication Configuration

```
// Full LDAP authentication setup
```

```
@Configuration
@EnableWebSecurity
public class LdapSecurityConfig {

@Value("${spring.ldap.urls}") private String ldapUrl;
@Value("${spring.ldap.base}") private String ldapBase;
@Value("${spring.ldap.username}") private String ldapUser;
@Value("${spring.ldap.password}") private String ldapPassword;

@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
return http
.authorizeHttpRequests(auth -> auth
.requestMatchers("/public/**").permitAll()
.anyRequest().authenticated())
.formLogin(Customizer.withDefaults())
.build();
}

@Bean
public AuthenticationManager authenticationManager() throws Exception {
LdapAuthenticationProviderConfigurer<AuthenticationManagerBuilder>
ldapConfig = new AuthenticationManagerBuilder(
new ObjectPostProcessorConfiguration()
.objectPostProcessor())
.ldapAuthentication();

ldapConfig
.userSearchBase("ou=Users")
.userSearchFilter("(uid={0})")
.groupSearchBase("ou=Groups")
.groupSearchFilter("(member={0})")
.rolePrefix("ROLE_")
.contextSource()
.url(ldapUrl + "/" + ldapBase)
.managerDn(ldapUser)
.managerPassword(ldapPassword);

return ldapConfig.and().build();
}
}
```

20.4 Active Directory Authentication

```
// ActiveDirectoryLdapAuthenticationProvider — simpler for AD
@Configuration
public class ActiveDirectoryConfig {

@Bean
public ActiveDirectoryLdapAuthenticationProvider adAuthProvider() {
ActiveDirectoryLdapAuthenticationProvider provider =
new ActiveDirectoryLdapAuthenticationProvider(
"yourcompany.com", // AD domain
"ldap://ad.yourcompany.com:389", // AD server
"DC=yourcompany,DC=com" // root DN
);

// Map AD groups to Spring Security roles
```

```
provider.setConvertSubErrorCodesToExceptions(true);
provider.setUseAuthenticationRequestCredentials(true);

// Custom search filter — {0}=username, {1}=domain
provider.setSearchFilter(
"(&(objectClass=user)(sAMAccountName={0}))");

return provider;
}

@Bean
public AuthenticationManager authManager() {
return new ProviderManager(adAuthProvider());
}
}
```

20.5 Map LDAP Groups to Roles

```
// Custom UserDetailsContextMapper — transform LDAP attrs
@Component
public class LdapUserDetailsMapper
implements UserDetailsContextMapper {

@Override
public UserDetails mapUserFromContext(DirContextOperations ctx,
String username,
Collection<? extends GrantedAuthority> authorities) {

String email = ctx.getStringAttribute("mail");
String name = ctx.getStringAttribute("cn");
String dept = ctx.getStringAttribute("departmentNumber");

// Map LDAP groups to application roles
List<GrantedAuthority> appAuthorities = authorities.stream()
.map(a -> mapGroupToRole(a.getAuthority()))
.filter(Objects::nonNull)
.map(SimpleGrantedAuthority::new)
.collect(Collectors.toList());

return CustomUserPrincipal.builder()
.username(username).email(email)
.firstName(extractFirst(name)).lastName(extractLast(name))
.tenantId(dept != null ? dept : "default")
.authorities(appAuthorities).enabled(true).build();
}

private String mapGroupToRole(String ldapGroup) {
return switch (ldapGroup.toLowerCase()) {
case "cn=admins,ou=groups" -> "ROLE_ADMIN";
case "cn=managers,ou=groups" -> "ROLE_MANAGER";
case "cn=employees,ou=groups" -> "ROLE_USER";
default -> null;
};
}

@Override
public void mapUserToContext(UserDetails user, DirContextAdapter ctx) {
```

```
throw new UnsupportedOperationException("LDAP write not supported");
}
}
```

Chapter 21

Spring Security Testing

21.1 Test Dependencies

```
<dependency>
<groupId>org.springframework.security</groupId>
<artifactId>spring-security-test</artifactId>
<scope>test</scope>
</dependency>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-test</artifactId>
<scope>test</scope>
</dependency>
```

21.2 MockMvc Security Testing

```
// Full MockMvc security test setup
@SpringBootTest
@AutoConfigureMockMvc
class SecurityIntegrationTest {

@Autowired MockMvc mockMvc;

// ■■ Public endpoints ■■■
```

```
@Test
@WithMockUser(roles = "ADMIN")
void adminEndpoint_adminRole_returns200() throws Exception {
mockMvc.perform(get("/api/admin/users"))
.andExpect(status().isOk());
}

// ■■ @WithMockUser with authorities ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
@Test
@WithMockUser(authorities = {"ROLE_USER", "order:write"})
void createOrder_withPermission_returns201() throws Exception {
mockMvc.perform(post("/api/orders")
.contentType(MediaType.APPLICATION_JSON)
.content("{"productId":1,"quantity":2}"))
.andExpect(status().isCreated());
}
}
```

21.3 Custom @WithMockUser Principal

```
// Test with your custom principal class
// 1. Define the factory
public class WithCustomUserSecurityContextFactory
implements WithSecurityContextFactory<WithCustomUser> {

@Override
public SecurityContext createSecurityContext(WithCustomUser annotation) {
CustomUserPrincipal principal = CustomUserPrincipal.builder()
.userId(annotation.userId())
.email(annotation.email())
.tenantId(annotation.tenantId())
.authorities(Arrays.stream(annotation.roles())
.map(r -> new SimpleGrantedAuthority("ROLE_" + r))
.collect(Collectors.toList()))
.build();

UsernamePasswordAuthenticationToken auth =
new UsernamePasswordAuthenticationToken(
principal, null, principal.getAuthorities());

SecurityContext context = SecurityContextHolder.createEmptyContext();
context.setAuthentication(auth);
return context;
}
}

// 2. Define the annotation
@Retention(RetentionPolicy.RUNTIME)
@WithSecurityContext(factory = WithCustomUserSecurityContextFactory.class)
public @interface WithCustomUser {
long userId() default 1L;
String email() default "user@test.com";
String tenantId() default "tenant-test";
String[] roles() default {"USER"};
}
```

```
// 3. Use in tests
@Test
@WithCustomUser(userId = 42L, email = "alice@acme.com",
tenantId = "acme", roles = {"ADMIN"})
void adminDashboard_customPrincipal_returns200() throws Exception {
mockMvc.perform(get("/api/admin/dashboard"))
.andExpect(status().isOk())
.andExpect(jsonPath("$.tenantId").value("acme"));
}
```

21.4 JWT Token Testing

```
// Test with real JWT tokens
@SpringBootTest @AutoConfigureMockMvc
class JwtSecurityTest {

@Autowired MockMvc mockMvc;
@Autowired JwtService jwtService;

private String generateTestToken(String role) {
CustomUserPrincipal principal = CustomUserPrincipal.builder()
.userId(1L).email("test@test.com").tenantId("test")
.authorities(List.of(new SimpleGrantedAuthority("ROLE_" + role)))
.build();
return jwtService.generateAccessToken(principal);
}

@Test
void withValidJwt_accessGranted() throws Exception {
String token = generateTestToken("USER");
mockMvc.perform(get("/api/orders")
.header("Authorization", "Bearer " + token))
.andExpect(status().isOk());
}

@Test
void withExpiredJwt_returns401() throws Exception {
String expired = generateExpiredToken();
mockMvc.perform(get("/api/orders")
.header("Authorization", "Bearer " + expired))
.andExpect(status().isUnauthorized())
.andExpect(jsonPath("$.code").value("TOKEN_EXPIRED"));
}

// Resource Server tests using jwt() request post-processor
@Test
void resourceServer_withJwtPostProcessor() throws Exception {
mockMvc.perform(get("/api/orders")
.with(jwt()
.jwt(j -> j.subject("1")
.claim("email", "test@test.com")
.claim("roles", List.of("USER")))))
.andExpect(status().isOk());
}
}
```

21.5 Method Security Testing

```
// Test @PreAuthorize on service methods directly
@SpringBootTest
class MethodSecurityTest {

@Autowired OrderService orderService;

@Test
@WithMockUser(roles = "USER")
void getOrder_owner_accessGranted() {
// Principal is "user" — @orderSecurity.isOwner checks match
assertThatNoException()
.isThrownBy(() -> orderService.getOrder(1L));
}

@Test
@WithMockUser(username = "other-user", roles = "USER")
void getOrder_notOwner_accessDenied() {
assertThatThrownBy(() -> orderService.getOrder(1L))
.isInstanceOf(AccessDeniedException.class);
}

@Test
@WithMockUser(roles = "ADMIN")
void getOrder_admin_bypassesOwnerCheck() {
assertThatNoException()
.isThrownBy(() -> orderService.getOrder(1L));
}

@Test
void getOrder_noAuth_throwsException() {
// No authentication in SecurityContext
assertThatThrownBy(() -> orderService.getOrder(1L))
.isInstanceOf(AuthenticationCredentialsNotFoundException.class);
}
}
```

Chapter 22

Reactive Security (WebFlux)

22.1 Reactive vs Servlet Security

Feature	Servlet (MVC)	Reactive (WebFlux)
Filter type	javax.servlet.Filter	WebFilter
Security config	HttpSecurity	ServerHttpSecurity
Auth principal	SecurityContextHolder	ReactiveSecurityContextHolder
UserDetailsService	UserDetailsService	ReactiveUserDetailsService
Password encoder	PasswordEncoder	PasswordEncoder (same)
Method security	@EnableMethodSecurity	@EnableReactiveMethodSecurity
JWT decoder	JwtDecoder	ReactiveJwtDecoder

22.2 WebFlux Security Configuration

```
// Reactive SecurityWebFilterChain
@Configuration
@EnableWebFluxSecurity
@EnableReactiveMethodSecurity
public class ReactiveSecurityConfig {

@Bean
public SecurityWebFilterChain springWebFilterChain(
ServerHttpSecurity http,
ReactiveJwtDecoder jwtDecoder) {

return http
.csrf(csrf -> csrf.disable())
.authorizeExchange(exchanges -> exchanges
.pathMatchers("/api/auth/**", "/api/public/**").permitAll()
.pathMatchers(HttpMethod.OPTIONS).permitAll()
.pathMatchers("/api/admin/**").hasRole("ADMIN")
.anyExchange().authenticated()
)
.oauth2ResourceServer(oauth2 -> oauth2
.jwt(jwt -> jwt
.jwtDecoder(jwtDecoder)
.jwtAuthenticationConverter(jwtAuthConverter()))
)
.exceptionHandling(ex -> ex
.authenticationEntryPoint(authEntryPoint())
.accessDeniedHandler(accessDeniedHandler())
)
.build();
}

@Bean
```

```
public ReactiveJwtDecoder jwtDecoder() {
return NimbusReactiveJwtDecoder
.withJwkSetUri("https://auth.yourapp.com/.well-known/jwks.json")
.build();
}
}
```

22.3 Reactive UserDetailsService

```
// Returns Mono instead of UserDetails
@Service
@RequiredArgsConstructor
public class ReactiveUserDetailsServiceImpl
implements ReactiveUserDetailsService {

private final UserRepository userRepository; // reactive R2DBC repo

@Override
public Mono<UserDetails> findByUsername(String email) {
return userRepository.findByEmail(email)
.switchIfEmpty(Mono.error(
new UsernameNotFoundException("Invalid credentials")))
.map(this::buildUserDetails);
}

private UserDetails buildUserDetails(User user) {
return org.springframework.security.core.userdetails.User
.withUsername(user.getEmail())
.password(user.getPassword())
.roles(user.getRoles().toArray(String[]::new))
.disabled(!user.isEnabled())
.accountLocked(user.isAccountLocked())
.build();
}
}
```

22.4 Accessing Current User in WebFlux

```
// ReactiveSecurityContextHolder — Mono-based
@RestController
@RequestMapping("/api")
public class ReactiveUserController {

// Option 1: inject via @AuthenticationPrincipal (same as MVC)
@GetMapping("/me")
public Mono<UserDto> getMe(
@AuthenticationPrincipal Mono<UserDetails> principal) {
return principal.map(u -> new UserDto(u.getUsername()));
}

// Option 2: ReactiveSecurityContextHolder (works anywhere)
@GetMapping("/profile")
public Mono<UserDto> getProfile() {
return ReactiveSecurityContextHolder.getContext()
.map(ctx -> ctx.getAuthentication().getPrincipal())
.cast(CustomUserPrincipal.class)
```

```
.map(UserDto::from);
}

// Option 3: In a service class
@Service
public class OrderService {
public Mono<List<OrderDto>> getMyOrders() {
return ReactiveSecurityContextHolder.getContext()
.map(ctx -> (CustomUserPrincipal)
ctx.getAuthentication().getPrincipal())
.flatMap(p -> orderRepository.findByUserId(p.getUserId())
.map(OrderDto::from)
.collectList());
}
}
}
```

22.5 Reactive JWT Filter

```
// Custom WebFilter for JWT — reactive equivalent of OncePerRequestFilter
@Component
@RequiredArgsConstructor
public class ReactiveJwtAuthFilter implements WebFilter {

private final ReactiveJwtDecoder jwtDecoder;
private final TokenBlacklistReactiveService blacklist;

@Override
public Mono<Void> filter(ServerWebExchange exchange, WebFilterChain chain) {
return Mono.justOrEmpty(extractToken(exchange.getRequest()))
.flatMap(token -> jwtDecoder.decode(token)
.flatMap(jwt -> {
String jti = jwt.getId();
return blacklist.isBlacklisted(jti)
.flatMap(blacklisted -> {
if (blacklisted)
return unauthorized(exchange, "Token revoked");

List<GrantedAuthority> authorities =
extractAuthorities(jwt);
Authentication auth =
new JwtAuthenticationToken(jwt, authorities);

return chain.filter(exchange)
.contextWrite(ReactiveSecurityContextHolder
.withAuthentication(auth));
});
})
.onErrorResume(JwtException.class, e ->
unauthorized(exchange, "Invalid token"))
)
.switchIfEmpty(chain.filter(exchange));
}
}
```

Chapter 23

Enterprise Security Patterns

23.1 RBAC — Role-Based Access Control

RBAC assigns permissions to roles, and roles to users. It's the most common enterprise access control model — simple to understand, audit, and maintain.

```
// Clean RBAC with role hierarchy
// Role hierarchy: ADMIN > MANAGER > USER > GUEST
@Bean public RoleHierarchy roleHierarchy() {
RoleHierarchyImpl h = new RoleHierarchyImpl();
h.setHierarchy("ROLE_ADMIN > ROLE_MANAGER\nROLE_MANAGER > ROLE_USER\nROLE_USER > ROLE_GUEST");
return h;
}

// Fine-grained permissions within roles
public enum Permission {
USER_READ("user:read"), USER_WRITE("user:write"),
ORDER_READ("order:read"), ORDER_WRITE("order:write"),
REPORT_EXPORT("report:export"), ADMIN_ALL("admin:*");
}

// Service layer enforces at permission level
@PreAuthorize("hasAuthority('order:write')")
public OrderDto createOrder(CreateOrderRequest req) { ... }

@PreAuthorize("hasAuthority('report:export') and hasRole('MANAGER')")
public byte[] exportReport(Long id) { ... }
```

23.2 ABAC — Attribute-Based Access Control

ABAC makes decisions based on attributes of the subject, resource, action, and environment. It enables fine-grained, context-aware policies that RBAC cannot express.

```
// ABAC policy examples
// Policy 1: User can only edit documents they own AND in same department
public boolean canEdit(Long docId, Authentication auth) {
CustomUserPrincipal user = (CustomUserPrincipal) auth.getPrincipal();
Document doc = documentRepo.findById(docId).orElseThrow();
return doc.getOwnerId().equals(user.getUserId()) &&
doc.getDeptId().equals(user.getDepartmentId());
}

// Policy 2: Financial operations only during business hours + MFA
public boolean canApprovePayment(Authentication auth) {
LocalTime now = LocalTime.now();
boolean inHours = now.isAfter(LocalTime.of(9,0))
&& now.isBefore(LocalTime.of(17,0));
boolean hasMfa = ((CustomUserPrincipal) auth.getPrincipal()).isMfaVerified();
return inHours && hasMfa;
}

// Policy 3: Clearance-based access to classified resources
public boolean canAccessResource(String classification, Authentication auth) {
```

```
int required = switch (classification) {
case "SECRET" -> 3;
case "CONFIDENTIAL" -> 2;
case "INTERNAL" -> 1;
default -> 0;
};
return ((CustomUserPrincipal) auth.getPrincipal())
.getClearanceLevel() >= required;
}
```

23.3 MFA — Multi-Factor Authentication

```
// TOTP-based MFA (Google Authenticator compatible)
// Dependencies: com.warrenstrange:googleauth:1.5.0

@Service @RequiredArgsConstructor
public class MfaService {

private final GoogleAuthenticator gAuth = new GoogleAuthenticator();
private final UserRepository userRepository;

// Step 1: Generate secret + QR code URI for enrolment
public MfaSetupResponse setupMfa(Long userId) {
GoogleAuthenticatorKey key = gAuth.createCredentials();
String secret = key.getKey();

// Store secret (encrypted) with user
userRepository.updateMfaSecret(userId,
encryptionService.encrypt(secret));

String qrUri = GoogleAuthenticatorQRGenerator.getOtpAuthTotpURL(
"YourApp", "user@yourapp.com", key);

return new MfaSetupResponse(secret, qrUri);
}

// Step 2: Verify TOTP code during login
public boolean verifyTotp(Long userId, int totpCode) {
String secret = encryptionService.decrypt(
userRepository.getMfaSecret(userId));
return gAuth.authorize(secret, totpCode);
}

// Step 3: Mark MFA verified in JWT claims
public String generateMfaToken(CustomUserPrincipal principal,
boolean mfaVerified) {
return Jwts.builder()
.subject(String.valueOf(principal.getUserId()))
.claim("mfaVerified", mfaVerified)
// ... other claims
.signWith(signingKey).compact();
}
}
```

23.4 SSO — Single Sign-On

```
// Spring Security acts as OIDC client for SSO
```

```
# application.yml — OIDC SSO via Keycloak
spring.security.oauth2.client:
registration:
keycloak:
client-id: my-app
client-secret: ${KEYCLOAK_SECRET}
scope: openid, profile, email, roles
authorization-grant-type: authorization_code
provider:
keycloak:
issuer-uri: https://auth.yourcompany.com/realms/corporate

// Result: Single login at Keycloak works across ALL registered apps
// User logs into App A → Keycloak session created
// User opens App B → Keycloak detects session → auto-authenticated
// User logs out → session terminated across ALL apps (OIDC RP-Initiated Logout)
```

23.5 Zero Trust Architecture Principles

- **Never trust, always verify** — authenticate and authorise every request, even internal ones
- **Least privilege** — grant minimum permissions needed; use fine-grained authorities
- **Assume breach** — design assuming the network is already compromised; encrypt everything
- **Explicit verification** — use MFA, device certificates, context-aware policies
- **Micro-segmentation** — enforce mTLS between services; no implicit internal trust
- **Continuous monitoring** — log all access decisions; alert on anomalies
- **Short-lived credentials** — JWT TTL 15 min, S2S tokens 30s, rotate secrets regularly

23.6 Pattern Comparison

Pattern	Best for	Spring Implementation
RBAC	Clear org hierarchy, simple access model	hasRole(), @Secured, role hierarchy
ABAC	Dynamic context-aware rules, compliance	PolicyEngine, AOP @Aspect, custom SpEL
MFA	High-value accounts, sensitive operations	TOTP + JWT claim mfaVerified
SSO	Multi-app enterprise, one login for all	OIDC client, Keycloak/Okta
Zero Trust	Cloud-native, microservices, post-breach	mTLS, S2S JWT, short TTLs, audit all

Chapter 24

Performance & Best Practices

24.1 Stateless Authentication

Stateless JWT authentication eliminates server-side session storage, enabling horizontal scaling with zero session affinity. Every request is self-contained — no DB lookup for session state.

```
// Stateless config — horizontal scaling ready
http.sessionManagement(s -> s
.sessionCreationPolicy(SessionCreationPolicy.STATELESS)
// No session = no sticky sessions = any node handles any request
)
.csrf(csrf -> csrf.disable()); // stateless = no CSRF needed

// JWT claims carry all state needed per request:
// userId, tenantId, roles, permissions, mfaVerified
// No DB call needed until you hit business logic
```

24.2 Caching User Details

```
// Cache UserDetails to avoid DB hit on every request
@Configuration
@EnableCaching
public class CacheConfig {

@Bean
public CacheManager cacheManager() {
// Redis cache for distributed deployments
RedisCacheConfiguration config = RedisCacheConfiguration
.defaultCacheConfig()
.entryTtl(Duration.ofMinutes(10)) // evict after 10 min
.disableCachingNullValues();

return RedisCacheManager.builder(redisConnectionFactory)
.withCacheConfiguration("userDetails", config)
.withCacheConfiguration("userDetailsById",
config.entryTtl(Duration.ofMinutes(5)))
.build();
}
}

// UserDetailsService with cache
@Cacheable(value = "userDetails", key = "#email",
unless = "#result == null")
public UserDetails loadUserByUsername(String email) {
// Only hits DB on cache miss (~first login per 10 min window)
return userRepository.findByEmailWithRoles(email)
.map(this::buildPrincipal).orElseThrow();
}

// Evict on any security change
@CacheEvict(value = {"userDetails","userDetailsById"}, allEntries = true)
public void evictAllUserCaches() { }
```

24.3 Database Query Optimisation

```
// Avoid N+1: load roles + permissions in one query
// WRONG — triggers N+1 (one query per role for permissions)
User user = userRepository.findByEmail(email).orElseThrow();
user.getRoles().forEach(role ->
role.getPermissions().size()); // lazy load = N extra queries

// CORRECT — single JOIN FETCH
@Query("SELECT DISTINCT u FROM User u LEFT JOIN FETCH u.roles r LEFT JOIN FETCH r.permissions WHERE
u.email = :email")
Optional<User> findByEmailWithRoles(@Param("email") String email);

// Index critical auth columns
CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_users_tenant ON users(tenant_id);
CREATE INDEX idx_jwt_blacklist ON jwt_blacklist(jti) WHERE revoked = true;
```

24.4 Secure Logging Best Practices

```
// Log security events -- never log sensitive data
@Slf4j
@Service
public class SecureLoggingService {

// CORRECT — log identifiers and actions, never credentials
public void logLoginSuccess(String email, String ip) {
log.info("LOGIN_SUCCESS email={} ip={} timestamp={}",
email, ip, Instant.now());
}

public void logLoginFailure(String email, String ip, String reason) {
log.warn("LOGIN_FAILURE email={} ip={} reason={}",
email, ip, reason);
}

// WRONG — never log passwords, tokens, or secrets
// log.info("User {} logged in with password {}", email, password);
// log.debug("JWT token: {}", token);
// log.info("API key: {}", apiKey);

// Mask sensitive fields in structured logs
public String maskEmail(String email) {
if (!StringUtils.hasText(email)) return "***";
int atIdx = email.indexOf('@');
if (atIdx <= 1) return "***@" + email.substring(atIdx + 1);
return email.charAt(0) + "***" + email.substring(atIdx);
// alice@example.com → a***@example.com
}
}
```

24.5 Performance Checklist

Concern	Problem	Solution
Session state	Session affinity, memory pressure	Stateless JWT — SessionCreationPolicy.STATELESS

Concern	Problem	Solution
DB per request	N+1 on roles/permissions load	JOIN FETCH query, Redis cache on UserDetailsService
BCrypt cost	Too high = slow registration	Cost 12 (~400ms) — tune to target 250-500ms
JWT size	Large tokens increase network overhead	Keep claims minimal — embed only userId, roles, tenantId
Blacklist lookup	Redis hit on every authenticated request	Set Redis TTL = token expiry — auto-cleanup
CORS preflight	OPTIONS on every request if not cached	Set maxAge=3600 — browser caches for 1 hour
@PreAuthorize AOP	Proxy overhead on every method call	Use URL-level rules for coarse gates, method security for fine-grained

Chapter 25

Frontend Integration

25.1 Token Storage Strategy

Storage	XSS Risk	CSRF Risk	Best for
localStorage	High — JS readable	None	Dev/low security
sessionStorage	High — JS readable	None	Single tab apps
Memory (React state)	Low — no persistence	None	High security SPAs
HttpOnly Cookie	None — JS cannot read	Yes — use CSRF	Traditional web + SPA
HttpOnly SameSite=Strict	None	Very low	Best overall

25.2 React — Secure Auth Implementation

```
// React auth with in-memory token storage
// AuthContext.tsx — tokens stored in memory, refresh token in httpOnly cookie
import { createContext, useContext, useRef, useState } from 'react';

const AuthContext = createContext(null);

export function AuthProvider({ children }) {
// Access token in memory — cleared on tab close, safe from XSS
const accessTokenRef = useRef(null);
const [user, setUser] = useState(null);

const login = async (email, password) => {
const res = await fetch('/api/auth/login', {
method: 'POST',
headers: { 'Content-Type': 'application/json' },
credentials: 'include', // allows httpOnly refresh token cookie
body: JSON.stringify({ email, password })
});
const data = await res.json();
accessTokenRef.current = data.accessToken; // in memory only
setUser(data.user);
};

const getToken = () => accessTokenRef.current;

const refreshToken = async () => {
// Refresh token sent automatically via httpOnly cookie
const res = await fetch('/api/auth/refresh', {
method: 'POST',
credentials: 'include' // sends the refresh token cookie
});
const data = await res.json();
accessTokenRef.current = data.accessToken;
return data.accessToken;
```

```
};

return (
<AuthContext.Provider value={{ user, login, getToken, refreshToken }}>
{children}
</AuthContext.Provider>
);
}
```

25.3 Angular — HTTP Interceptor

```
// Angular interceptor with refresh queue
@Injectable()
export class JwtInterceptor implements HttpInterceptor {

private isRefreshing = false;
private refreshSubject = new BehaviorSubject<string | null>(null);

constructor(private auth: AuthService) {}

intercept(req: HttpRequest<any>, next: HttpHandler) {
return next.handle(this.addToken(req)).pipe(
catchError(err => {
if (err instanceof HttpErrorResponse && err.status === 401) {
return this.handle401(req, next);
}
return throwError(() => err);
})
);
}

private addToken(req: HttpRequest<any>) {
const token = this.auth.getAccessToken();
return token
? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } })
: req;
}

private handle401(req: HttpRequest<any>, next: HttpHandler) {
if (!this.isRefreshing) {
this.isRefreshing = true;
this.refreshSubject.next(null);

return this.auth.refresh().pipe(
switchMap(token => {
this.isRefreshing = false;
this.refreshSubject.next(token);
return next.handle(this.addToken(req));
}),
catchError(() => { this.auth.logout(); return EMPTY; })
);
}
return this.refreshSubject.pipe(
filter(t => t !== null), take(1),
switchMap(() => next.handle(this.addToken(req)))
);
}
```

```
}
```

25.4 React Native / Mobile

```
// Secure token storage on mobile
// React Native — use Keychain / Keystore (never AsyncStorage for tokens)
import * as Keychain from 'react-native-keychain';

// Store tokens securely
await Keychain.setGenericPassword(
'tokens',
JSON.stringify({ accessToken, refreshToken }),
{ accessControl: Keychain.ACCESS_CONTROL.BIOMETRY_ANY_OR_DEVICE_PASSCODE,
accessible: Keychain.ACCESSIBLE.WHEN_UNLOCKED_THIS_DEVICE_ONLY }
);

// Retrieve
const creds = await Keychain.getGenericPassword();
const { accessToken, refreshToken } = JSON.parse(creds.password);

// iOS: stores in Keychain (hardware-backed)
// Android: stores in Keystore (hardware-backed if available)
// NEVER use: AsyncStorage, localStorage — plain text on device
```

Chapter 26

Logging & Monitoring

26.1 Security Audit Events

```
// Listen to Spring Security application events
@Component @Slf4j
public class SecurityAuditListener {

@EventListener
public void onAuthSuccess(AuthenticationSuccessEvent event) {
String user = event.getAuthentication().getName();
log.info("AUDIT LOGIN_SUCCESS user={}", user);
}

@EventListener
public void onAuthFailure(AbstractAuthenticationFailureEvent event) {
String user = event.getAuthentication().getName();
log.warn("AUDIT LOGIN_FAILURE user={} reason={}",
user, event.getException().getMessage());
}

@EventListener
public void onAccessDenied(AuthorizationDeniedEvent<?> event) {
String user = event.getAuthentication().get().getName();
log.warn("AUDIT ACCESS_DENIED user={}", user);
}

@EventListener
public void onLogout(LogoutSuccessEvent event) {
log.info("AUDIT LOGOUT user={}",
event.getAuthentication().getName());
}
}
```

26.2 Structured Security Logging

```
// logback-spring.xml — JSON structured logs
<configuration>
<appender name="JSON" class="ch.qos.logback.core.ConsoleAppender">
<encoder class="net.logstash.logback.encoder.LogstashEncoder">
<includeMdcKeyName>userId</includeMdcKeyName>
<includeMdcKeyName>tenantId</includeMdcKeyName>
<includeMdcKeyName>requestId</includeMdcKeyName>
<includeMdcKeyName>traceId</includeMdcKeyName>
<customFields>{"service":"order-service","env":"prod"}</customFields>
</encoder>
</appender>
<root level="INFO">
<appender-ref ref="JSON"/>
</root>
</configuration>

// Example log output (JSON):
```

```
{
"timestamp": "2025-01-15T10:30:00Z",
"level": "WARN",
"message": "AUDIT ACCESS_DENIED",
"userId": "42",
"tenantId": "acme",
"requestId": "uuid-1234",
"traceId": "trace-5678",
"service": "order-service"
}
```

26.3 Suspicious Activity Detection

```
// Detect and alert on anomalous patterns
@Service @RequiredArgsConstructor @Slf4j
public class SecurityMonitoringService {

private final RedisTemplate<String, String> redis;
private final AlertService alertService;

// Detect brute-force: > 5 failures in 10 min window
public void recordFailedLogin(String email, String ip) {
String key = "fail:" + email + ":" + ip;
Long count = redis.opsForValue().increment(key);
redis.expire(key, Duration.ofMinutes(10));

if (count != null && count >= 5) {
alertService.sendSecurityAlert(
"BRUTE_FORCE_DETECTED",
Map.of("email", email, "ip", ip, "count", count));
log.error("SECURITY BRUTE_FORCE email={} ip={} count={}",
email, ip, count);
}
}

// Detect impossible travel: login from 2 countries in < 1hr
public void checkTravelAnomaly(String userId, String country,
Instant loginTime) {
String lastKey = "last_login:" + userId;
String last = redis.opsForValue().get(lastKey);

if (last != null) {
LoginRecord prev = deserialize(last);
long minutesBetween = Duration.between(
prev.getTime(), loginTime).toMinutes();

if (!prev.getCountry().equals(country) && minutesBetween < 60) {
alertService.sendSecurityAlert("IMPOSSIBLE_TRAVEL",
Map.of("userId", userId,
"prevCountry", prev.getCountry(),
"newCountry", country,
"minutesApart", minutesBetween));
}
}
redis.opsForValue().set(lastKey,
serialize(new LoginRecord(country, loginTime)),
Duration.ofHours(2));
```

```
}
}
```

26.4 Metrics with Micrometer

```
// Security metrics for dashboards and alerting
@Component @RequiredArgsConstructor
public class SecurityMetrics {

private final MeterRegistry meterRegistry;

// Counters
private final Counter loginSuccess;
private final Counter loginFailure;
private final Counter tokenExpired;
private final Counter accessDenied;

@PostConstruct
public void init() {
loginSuccess = Counter.builder("security.login.success")
.description("Successful logins").register(meterRegistry);
loginFailure = Counter.builder("security.login.failure")
.description("Failed login attempts").register(meterRegistry);
tokenExpired = Counter.builder("security.token.expired")
.description("Expired JWT tokens").register(meterRegistry);
accessDenied = Counter.builder("security.access.denied")
.description("Access denied events").register(meterRegistry);
}

// Gauge: active sessions
@PostConstruct
public void registerGauges() {
Gauge.builder("security.sessions.active",
sessionRegistry, SessionRegistry::getAllPrincipals)
.description("Currently active sessions")
.register(meterRegistry);
}
}
```

Chapter 27

Cloud & DevOps Security

27.1 Docker Security

```
# Secure Dockerfile for Spring Boot
# Multi-stage build — keeps secrets out of final image
FROM eclipse-temurin:21-jdk-alpine AS builder
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn package -DskipTests

# Production image
FROM eclipse-temurin:21-jre-alpine

# Run as non-root user — never run containers as root
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
USER appuser

WORKDIR /app
COPY --from=builder /app/target/*.jar app.jar

# Don't expose secrets as ENV vars — use secrets management
EXPOSE 8080

# Read-only filesystem
# docker run --read-only --tmpfs /tmp ...

ENTRYPOINT ["java",
"-Djava.security.egd=file:/dev/./urandom",
"-Dspring.profiles.active=prod",
"-jar", "app.jar"]
```

```
# docker-compose.yml — security settings
services:
api:
image: your-app:latest
read_only: true # read-only container filesystem
tmpfs:
- /tmp # writable temp dir only
security_opt:
- no-new-privileges:true # process cannot escalate privileges
user: "1000:1000" # run as non-root
environment:
# Reference secrets — never hard-code values
- SPRING_DATASOURCE_PASSWORD_FILE=/run/secrets/db_password
secrets:
- db_password
cap_drop:
- ALL # drop all Linux capabilities
cap_add:
- NET_BIND_SERVICE # only add what's needed

secrets:
```

```
db_password:
external: true
```

27.2 Kubernetes Secrets

```
# k8s Secret — base64 encoded
```

```
# Create secret
kubectl create secret generic app-secrets \
--from-literal=DB_PASSWORD=supersecret \
--from-literal=JWT_SECRET=yoursecretkey \
-n production

# Reference in Deployment
spec:
containers:
- name: api
image: your-app:latest
env:
- name: DB_PASSWORD
valueFrom:
secretKeyRef:
name: app-secrets
key: DB_PASSWORD
- name: JWT_SECRET
valueFrom:
secretKeyRef:
name: app-secrets
key: JWT_SECRET
volumeMounts:
- name: secrets-volume
mountPath: /run/secrets
readOnly: true
```

```
# HashiCorp Vault integration — production grade
```

```
# application.yml — Spring Vault
spring:
cloud:
vault:
host: vault.yourcompany.com
port: 8200
scheme: https
authentication: KUBERNETES # uses k8s service account token
kubernetes:
role: spring-app
kv:
enabled: true
backend: secret
default-context: app/production

# Vault stores:
# secret/app/production/db.password
# secret/app/production/jwt.secret
# secret/app/production/ssl.keystore-password
# Spring Cloud Vault injects them as Spring properties automatically
```

27.3 CI/CD Pipeline Security

```
# GitHub Actions — secure pipeline
name: Build & Deploy
on: [push]
jobs:
security-scan:
runs-on: ubuntu-latest
steps:
# Dependency vulnerability scan
- uses: actions/checkout@v4
- name: OWASP Dependency Check
uses: dependency-check/Dependency-Check_Action@main
with:
project: 'spring-app'
format: 'SARIF'
failBuildOnCVSS: 7 # fail if critical CVE found

# Container image scan
- name: Trivy container scan
uses: aquasecurity/trivy-action@master
with:
image-ref: 'your-app:${{ github.sha }}'
severity: 'CRITICAL,HIGH'
exit-code: '1'

# Static code analysis
- name: SpotBugs Security Analysis
run: mvn spotbugs:check -Dspotbugs.plugins=com.h3xstream.findsecbugs

deploy:
needs: security-scan
environment: production # requires approval
steps:
- name: Deploy to k8s
env:
KUBECONFIG: ${{ secrets.KUBECONFIG }}
run: kubectl set image deployment/api api=your-app:${{ github.sha }}
```

27.4 Cloud Security Checklist

- Never run containers as root — use non-root user in Dockerfile
- Never store secrets in environment variables in Dockerfile or image
- Use secrets manager: HashiCorp Vault, AWS Secrets Manager, or k8s Secrets
- Enable read-only container filesystem — only /tmp writable via tmpfs
- Drop all Linux capabilities — only add NET_BIND_SERVICE if port < 1024
- Scan container images for CVEs in CI/CD pipeline (Trivy, Snyk)
- Enable RBAC on Kubernetes — least-privilege service accounts
- Rotate all secrets regularly — Vault dynamic secrets automate this

Chapter 28

Real-Time Security (WebSocket)

28.1 WebSocket Security Challenges

WebSockets are persistent connections that upgrade from HTTP. They don't send HTTP headers on each message, so standard filter-based auth does not apply after the handshake. Authentication must happen at the handshake, and authorisation at the message level.

```
// WebSocket security layers
// Layer 1: Handshake authentication (HTTP upgrade request)
// → JWT in query param or cookie (headers not available in browsers)
// Layer 2: Message-level authorisation
// → @PreAuthorize on @MessageMapping methods
// Layer 3: Destination-level access control
// → configureMessageBroker rules per topic prefix
```

28.2 WebSocket Security Configuration

```
// Full WebSocket + STOMP security setup
@Configuration
@EnableWebSocketMessageBroker
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {

@Override
public void configureMessageBroker(MessageBrokerRegistry registry) {
registry.enableSimpleBroker("/topic", "/queue");
registry.setApplicationDestinationPrefixes("/app");
registry.setUserDestinationPrefix("/user");
}

@Override
public void registerStompEndpoints(StompEndpointRegistry registry) {
registry.addEndpoint("/ws")
.setAllowedOrigins("https://app.yourapp.com")
.withSockJS();
}
}

@Configuration
public class WebSocketSecurityConfig
extends AbstractSecurityWebSocketMessageBrokerConfigurer {

@Override
protected void configureInbound(
MessageSecurityMetadataSourceRegistry messages) {
messages
// Public topics
.simpDestMatchers("/topic/public/**").permitAll()
// Private user queue — only for the user themselves
.simpSubscribeDestMatchers("/user/queue/**").authenticated()
// App destinations
.simpDestMatchers("/app/**").authenticated()
// Admin topics
```

```
.simpDestMatchers("/topic/admin/**").hasRole("ADMIN")
// Deny everything else
.anyMessage().denyAll();
}

@Override
protected boolean sameOriginDisabled() {
return false; // keep CSRF protection enabled
}
}
```

28.3 Handshake Authentication — JWT via Query Param

```
// Authenticate during WebSocket handshake
@Component @RequiredArgsConstructor
public class JwtHandshakeInterceptor
implements HandshakeInterceptor {

private final JwtService jwtService;

@Override
public boolean beforeHandshake(ServerHttpRequest request,
ServerHttpResponse response,
WebSocketHandler wsHandler,
Map<String, Object> attributes) {

// Extract JWT from query param (browsers can't set WS headers)
String query = request.getURI().getQuery();
String token = extractTokenFromQuery(query);

if (token == null) return false; // reject handshake

try {
Claims claims = jwtService.validateAndExtractClaims(token);
// Store user info in WebSocket session attributes
attributes.put("userId", claims.getSubject());
attributes.put("email", claims.get("email"));
attributes.put("tenantId", claims.get("tenantId"));
attributes.put("roles", claims.get("roles"));
return true; // allow handshake

} catch (Exception e) {
return false; // reject handshake — invalid token
}
}

@Override
public void afterHandshake(...) {}
}

// Register the interceptor
@Override
public void registerStompEndpoints(StompEndpointRegistry registry) {
registry.addEndpoint("/ws")
.addInterceptors(jwtHandshakeInterceptor)
.setAllowedOrigins("https://app.yourapp.com")
.withSockJS();
```

```
}
```

28.4 STOMP Message-Level Security

```
// @PreAuthorize on @MessageMapping methods
@Controller
@RequiredArgsConstructor
public class ChatController {

private final SimpMessagingTemplate messaging;

// Only authenticated users can send messages
@MessageMapping("/chat.send")
@PreAuthorize("isAuthenticated()")
public void sendMessage(ChatMessage message,
Principal principal) {
// Ensure sender matches authenticated user (prevent spoofing)
message.setSender(principal.getName());
message.setTimestamp(Instant.now());

// Broadcast to topic
messaging.convertAndSend("/topic/chat", message);
}

// Private messages — only to the intended user
@MessageMapping("/chat.private")
@PreAuthorize("isAuthenticated()")
public void privateMessage(PrivateMessage message,
Principal principal) {
message.setFrom(principal.getName());
// Send only to specific user's queue
messaging.convertAndSendToUser(
message.getTo(), "/queue/private", message);
}

// Admin-only broadcast
@MessageMapping("/admin.broadcast")
@PreAuthorize("hasRole('ADMIN')")
public void adminBroadcast(AdminMessage message) {
messaging.convertAndSend("/topic/admin", message);
}
}
```

Chapter 29

Advanced Topics

29.1 Custom Authentication Provider

```
// Full custom AuthenticationProvider
// Use case: authenticate via hardware token, biometric, OTP, or external service

@Component @RequiredArgsConstructor
public class OtpAuthenticationProvider
implements AuthenticationProvider {

private final OtpService otpService;
private final UserRepository userRepository;
private final PasswordEncoder encoder;

@Override
public Authentication authenticate(Authentication authentication)
throws AuthenticationException {

OtpAuthentication request = (OtpAuthentication) authentication;
String email = request.getPrincipal().toString();
String otp = request.getCredentials().toString();

User user = userRepository.findByEmail(email)
.orElseThrow(() ->
new UsernameNotFoundException("Invalid credentials"));

if (!otpService.verifyOtp(user.getId(), otp)) {
throw new BadCredentialsException("Invalid OTP");
}

if (!user.isEnabled()) throw new DisabledException("Account disabled");
if (user.isAccountLocked()) throw new LockedException("Account locked");

List<GrantedAuthority> authorities = buildAuthorities(user);

return new OtpAuthentication(
user.getEmail(), null, authorities);
}

@Override
public boolean supports(Class<?> authentication) {
return OtpAuthentication.class.isAssignableFrom(authentication);
}
}

// Wire multiple providers
@Bean
public AuthenticationManager authManager() {
return new ProviderManager(List.of(
daoAuthenticationProvider(), // username + password
otpAuthenticationProvider(), // OTP
apiKeyAuthenticationProvider() // API key
));
```

```
}
```

29.2 Dynamic Role Management

```
// Change roles at runtime without redeployment
@Service @RequiredArgsConstructor
public class DynamicRoleService {

private final UserRepository userRepository;
private final RoleRepository roleRepository;
private final CacheManager cacheManager;
private final SessionRegistry sessionRegistry;

@Transactional
public void assignRole(Long userId, String roleName) {
User user = userRepository.findById(userId).orElseThrow();
Role role = roleRepository.findByName(roleName).orElseThrow();

user.getRoles().add(role);
userRepository.save(user);

// Immediately invalidate caches
evictUserCaches(user.getEmail());

// Force re-login for running sessions
invalidateUserSessions(user.getEmail());

log.info("Role {} assigned to user {}", roleName, userId);
}

private void evictUserCaches(String email) {
cacheManager.getCache("userDetails").evict(email);
cacheManager.getCache("userDetailsById").clear();
}

private void invalidateUserSessions(String email) {
sessionRegistry.getAllPrincipals().stream()
.filter(p -> p instanceof UserDetails ud
&& ud.getUsername().equals(email))
.flatMap(p -> sessionRegistry.getAllSessions(p,false).stream())
.forEach(SessionInformation::expireNow);
}
}
```

29.3 Security Context Propagation — Virtual Threads (Java 21)

```
// Spring Boot 3.2+ — virtual threads with security
# application.yml — enable virtual threads
spring:
threads:
virtual:
enabled: true

// Spring Security automatically propagates SecurityContext
// to virtual threads via InheritableThreadLocal
```

```
// But for explicit async work, use DelegatingSecurityContextExecutor
@Bean
public Executor virtualThreadExecutor() {
ExecutorService virtual = Executors.newVirtualThreadPerTaskExecutor();

// Wrap with security context propagation
return new DelegatingSecurityContextExecutorService(virtual);
}

// Usage — SecurityContext available in virtual thread
@Async
@PreAuthorize("isAuthenticated()")
public CompletableFuture<Report> generateReport(Long reportId) {
// Runs on virtual thread — SecurityContext is propagated
String user = SecurityContextHolder.getContext()
.getAuthentication().getName();
return CompletableFuture.supplyAsync(() -> buildReport(reportId, user));
}
```

29.4 Security Observability

```
// OpenTelemetry distributed tracing for security events
@Component @RequiredArgsConstructor
public class SecurityTracer {

private final Tracer tracer;

public <T> T traceSecurityCheck(String operation,
Supplier<T> securedOp) {
Span span = tracer.spanBuilder("security." + operation)
.setAttribute("security.operation", operation)
.startSpan();

Authentication auth = SecurityContextHolder.getContext()
.getAuthentication();
if (auth != null) {
span.setAttribute("security.user",
auth.getName());
span.setAttribute("security.roles",
auth.getAuthorities().toString());
}

try (Scope scope = span.makeCurrent()) {
T result = securedOp.get();
span.setStatus(StatusCode.OK);
return result;
} catch (AccessDeniedException e) {
span.setStatus(StatusCode.ERROR, "ACCESS_DENIED");
span.recordException(e);
throw e;
} finally {
span.end();
}
}
}
```

Chapter 30

Final Enterprise Projects

30.1 Project 1 — JWT Authentication System

A production-grade stateless REST API with JWT authentication, refresh token rotation, Redis blacklisting, brute-force protection, and complete user lifecycle management.

Component	Technology	Purpose
Auth API	Spring Boot 3.x	Login, register, refresh, logout
JWT	JJWT 0.12.x + RS256	Stateless access tokens (15 min TTL)
Refresh tokens	PostgreSQL + Redis	Rotation + reuse detection
Rate limiting	Redis + Bucket4j	Brute-force protection
User store	PostgreSQL + JPA	Users, roles, permissions, history
Cache	Redis	UserDetails + token blacklist
Audit	AuditLog entity	All auth events persisted
Tests	JUnit 5 + MockMvc	Unit + integration test suite

```
// Project 1 — architecture summary
POST /api/auth/register → validate → hash password → save user → send email
POST /api/auth/verify-email → validate token → set emailVerified=true
POST /api/auth/login → authenticate → issue accessToken + refreshToken
POST /api/auth/refresh → rotate refreshToken → issue new accessToken
POST /api/auth/logout → blacklist accessToken → revoke refreshToken
POST /api/auth/forgot-password→ generate reset token → send email
POST /api/auth/reset-password → validate token → update password → evict cache
GET /api/users/me → @AuthenticationPrincipal → return profile
PUT /api/users/password → verify old → validate policy → hash new → save
```

30.2 Project 2 — Secure Microservices Platform

Enterprise microservices with API Gateway security, service-to-service JWT authentication, multi-tenancy, and distributed security audit logging.

Service	Port	Responsibility
API Gateway	8080	JWT validation, rate limiting, header injection
Auth Service	8081	Login, token issuance, user management
Order Service	8082	Order CRUD — scoped by tenant + user
Inventory Service	8083	Stock management — S2S authenticated
Payment Service	8084	Payments — MFA required, extra audit
Notification Svc	8085	Email/SMS — triggered by Kafka events
Audit Service	8086	Centralised security event store

```
// Security flow across services
Client → [JWT] → API Gateway
↓ validate JWT
↓ inject X-User-Id, X-Tenant-Id, X-User-Roles
↓ strip Authorization header
Order Service (reads headers → SecurityContext)
↓ @PreAuthorize("hasRole('USER')")
↓ tenantId-scoped DB queries
↓ calls Inventory via [X-Service-Token 30s]
Inventory Service (validates X-Service-Token)
↓ ROLE_SERVICE authority
↓ original user context propagated
↓ Kafka event → { payload, userId, tenantId }
Notification Service (restores SecurityContext from event)
↓ sends email with correct tenant branding
```

30.3 Project 3 — Banking Security Application

High-security financial application with MFA, ABAC policy engine, comprehensive audit trail, PCI-DSS compliance headers, and anomaly detection.

Security Feature	Implementation	Compliance
Authentication	Password + TOTP MFA	PCI-DSS Req 8
Passwords	PBKDF2 (600k iter, SHA-512) + pepper	NIST SP 800-63B
Session	15-min JWT, 30-min inactivity timeout	PCI-DSS Req 8
Authorisation	ABAC policy engine + role hierarchy	Principle of least privilege
Transport	TLS 1.3 only, HSTS preload	PCI-DSS Req 4
Audit	Immutable audit log, all events	PCI-DSS Req 10
Anomaly detection	Impossible travel, velocity checks	PCI-DSS Req 10.6
DB security	Encrypted at rest, least-privilege user	PCI-DSS Req 3, 7

30.4 Learning Path Completion

Level	Topics Mastered	Chapters
Beginner	Spring Security basics, authentication, authorization	1, 2, 3, 4, 5
Intermediate	JWT, filters, method security, DB security, testing	6, 7, 8, 15, 16, 21
Advanced	OAuth2, OIDC, microservices, reactive, WebSocket	9, 10, 11, 22, 28
Enterprise	SSO, MFA, Zero Trust, cloud, monitoring, ABAC	12-14, 17-20, 23-27, 29, 30

30.5 Recommended Tech Stack

Category	Technology	Version
Language	Java	21 LTS
Framework	Spring Boot	3.3.x

Category	Technology	Version
Security	Spring Security	6.3.x
Build	Maven / Gradle	3.9 / 8.x
Database	PostgreSQL	16.x
Cache/Store	Redis	7.x
Message Bus	Apache Kafka	3.7.x
Observability	Micrometer + OTEL	Latest
Testing	JUnit 5 + MockMvc	Spring Boot Starter Test
Auth Server	Keycloak	24.x
Container	Docker + k8s	Latest
Secrets	HashiCorp Vault	1.17.x